

Abstract Data Type

ADT Coda

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio

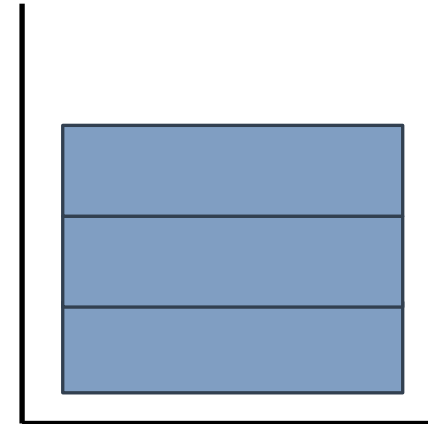


Coda (Queue)

Coda (Queue)



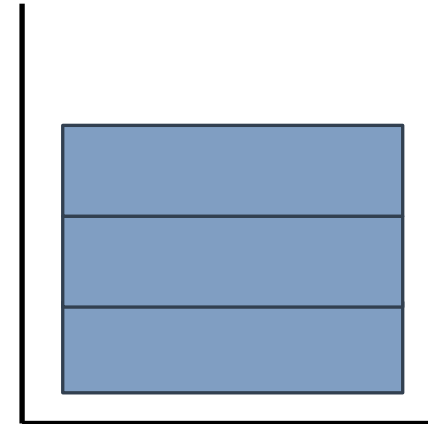
- Una coda (queue) è:
 - una catena di elementi dello stesso tipo
 - Esempio:
 - Coda di persone in biglietteria
-
-
-
-



Coda (Queue)

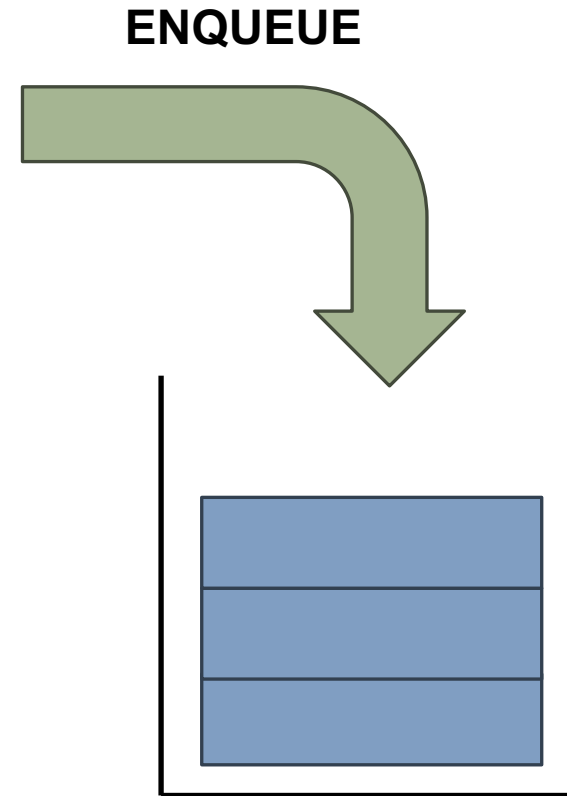


- Una coda (queue) è:
 - una catena di elementi dello stesso tipo
 - Esempio:
 - Coda di persone in biglietteria
- Una coda è gestita
 - Seguendo la politica FIFO (First-In-First-Out)
 -
 -



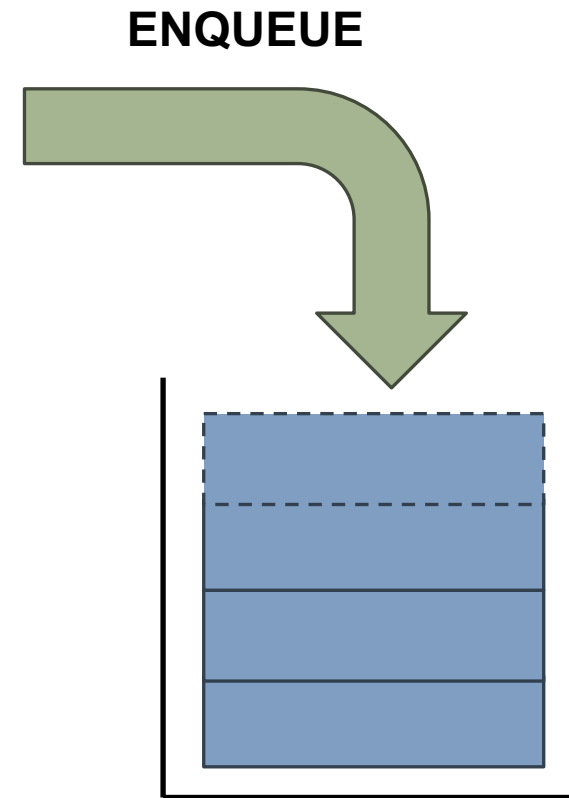
Coda (Queue)

- Una coda (queue) è:
 - una catena di elementi dello stesso tipo
 - Esempio:
 - Coda di persone in biglietteria
- Una coda è gestita
 - Seguendo la politica FIFO (First-In-First-Out)
 - Aggiungo in coda: **ENQUEUE**
 -



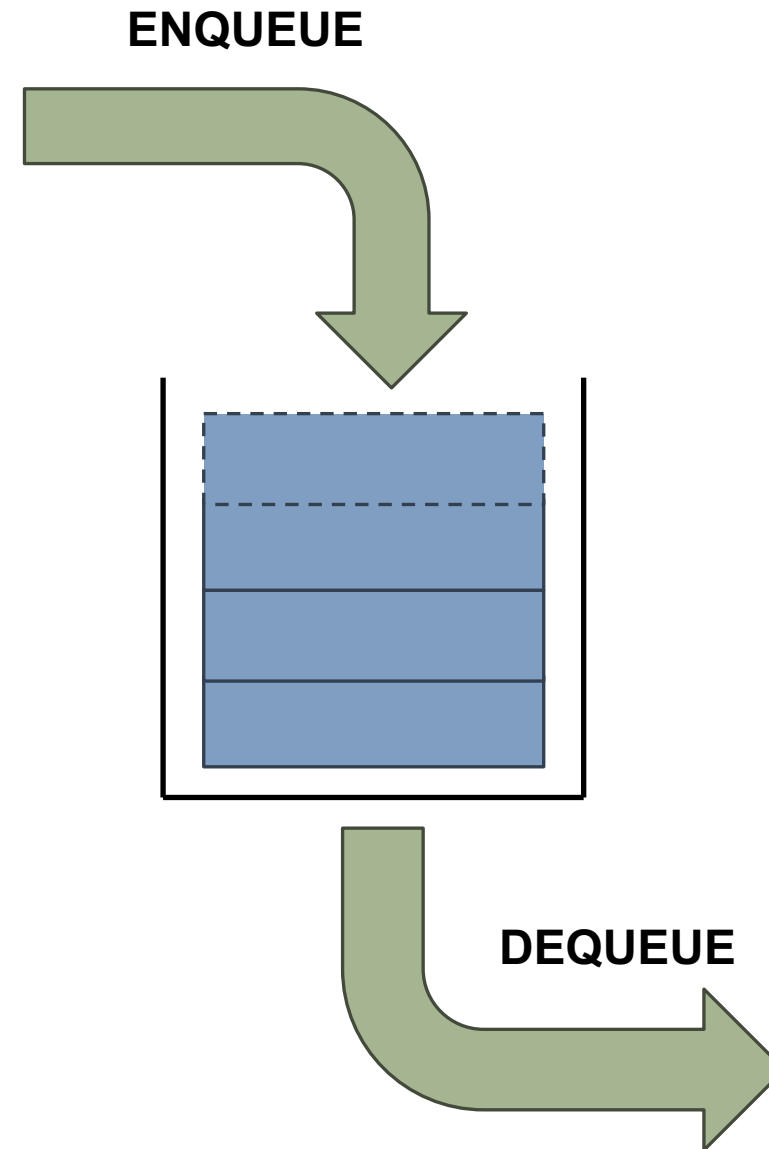
Coda (Queue)

- Una coda (queue) è:
 - una catena di elementi dello stesso tipo
 - Esempio:
 - Coda di persone in biglietteria
- Una coda è gestita
 - Seguendo la politica FIFO (First-In-First-Out)
 - Aggiungo in coda: **ENQUEUE**
 -



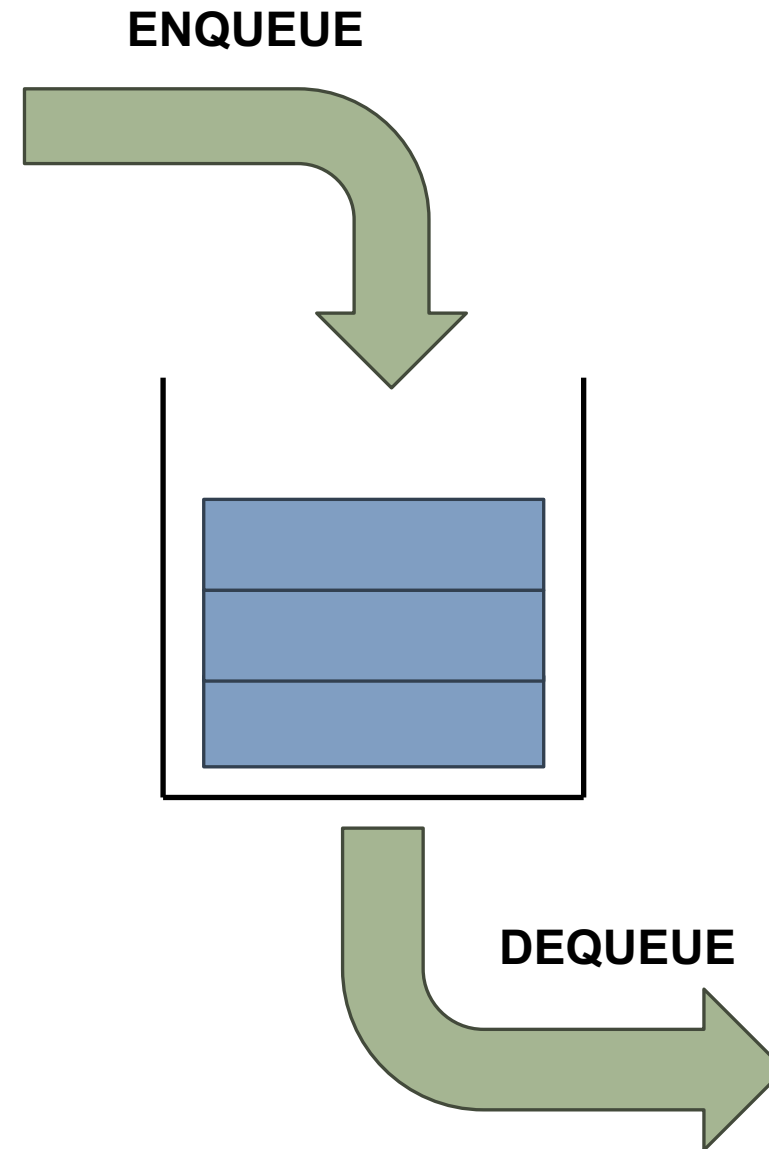
Coda (Queue)

- Una coda (queue) è:
 - una catena di elementi dello stesso tipo
 - Esempio:
 - Coda di persone in biglietteria
- Una coda è gestita
 - Seguendo la politica FIFO (First-In-First-Out)
 - Aggiungo in coda: **ENQUEUE**
 - Tolgo in cima (primo inserito): **DEQUEUE**

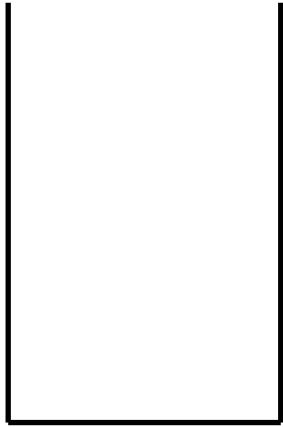


Coda (Queue)

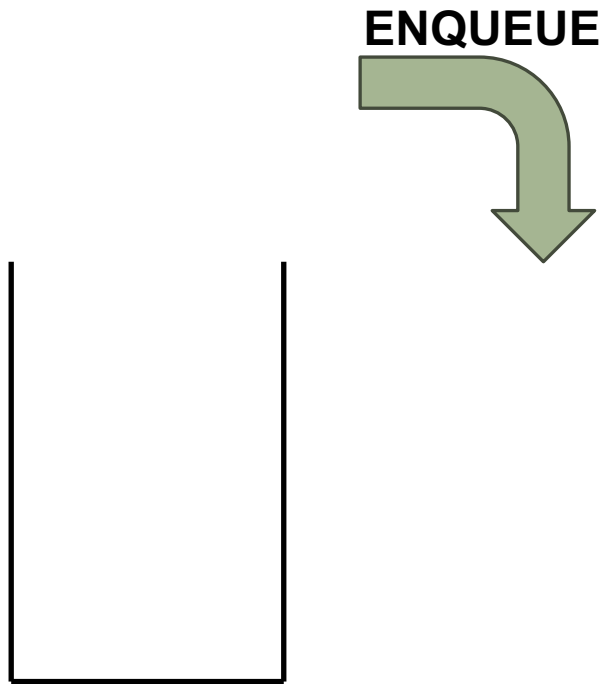
- Una coda (queue) è:
 - una catena di elementi dello stesso tipo
 - Esempio:
 - Coda di persone in biglietteria
- Una coda è gestita
 - Seguendo la politica FIFO (First-In-First-Out)
 - Aggiungo in coda: **ENQUEUE**
 - Tolgo in cima (primo inserito): **DEQUEUE**



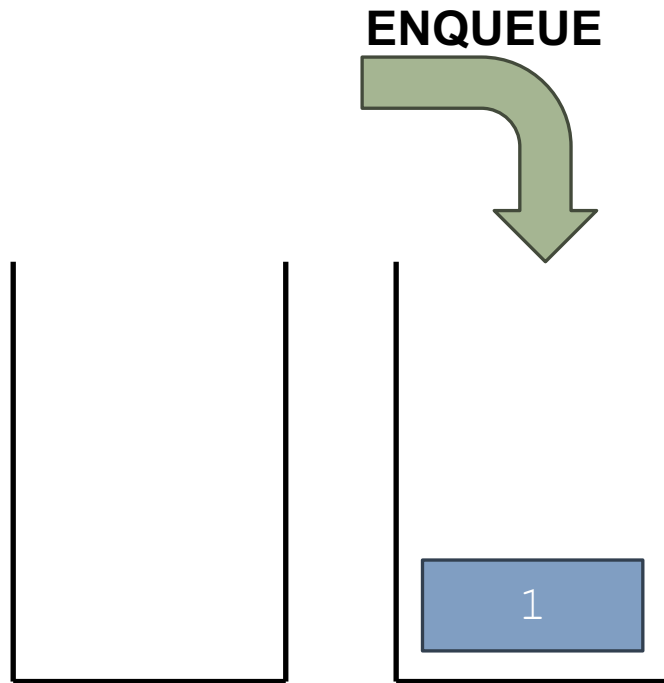
Coda (Queue): Esempio



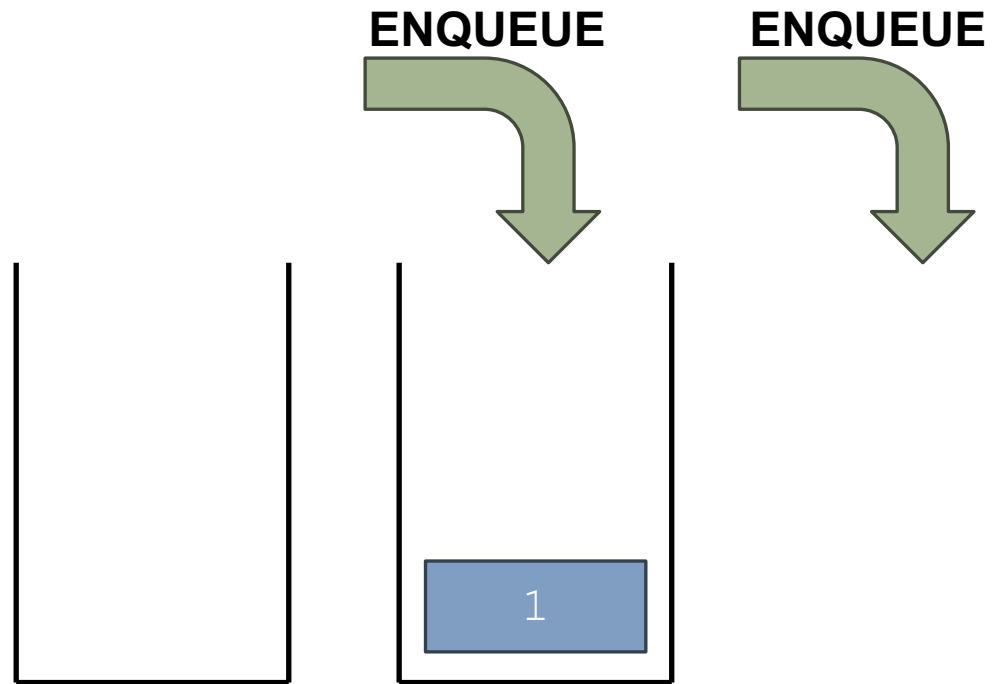
Coda (Queue): Esempio



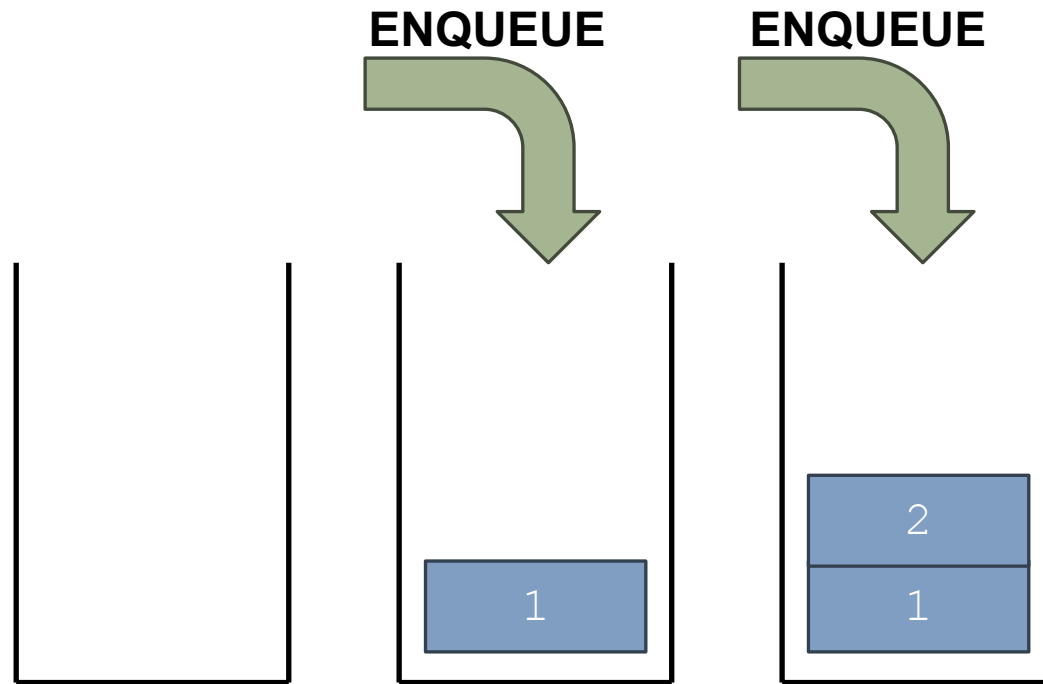
Coda (Queue): Esempio



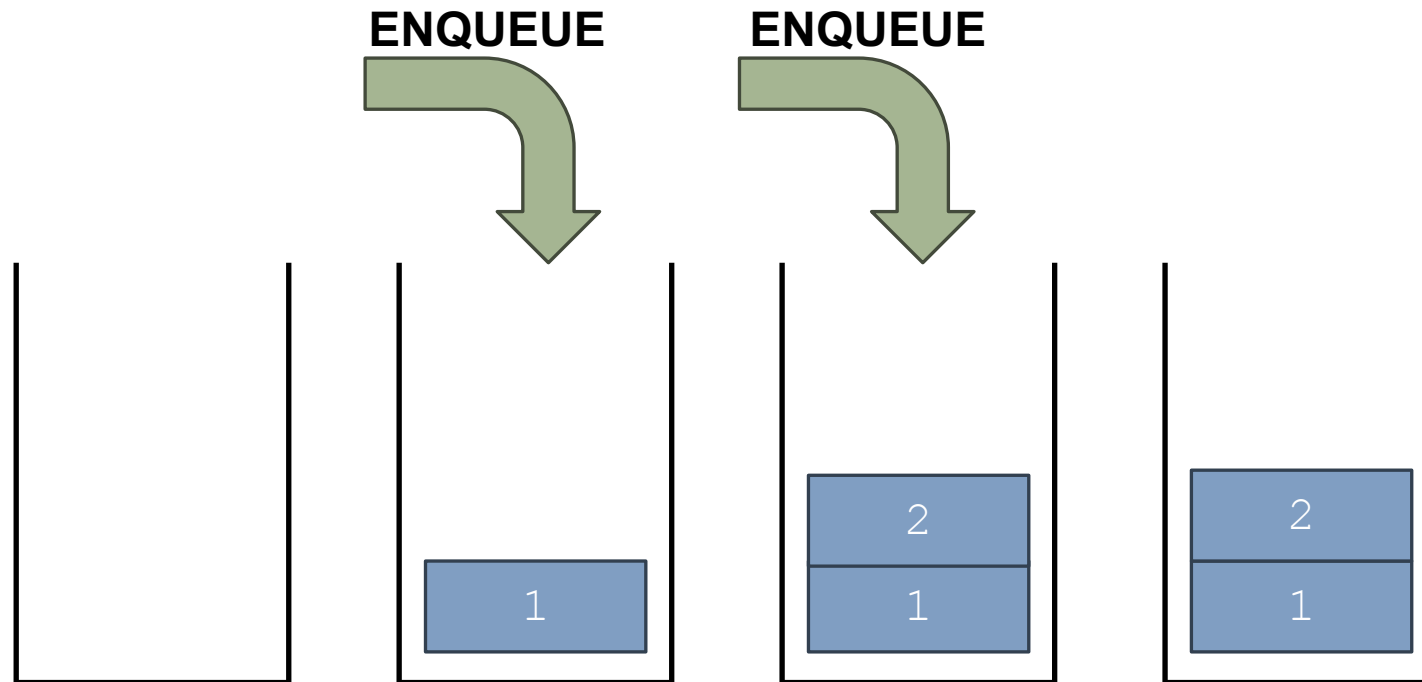
Coda (Queue): Esempio



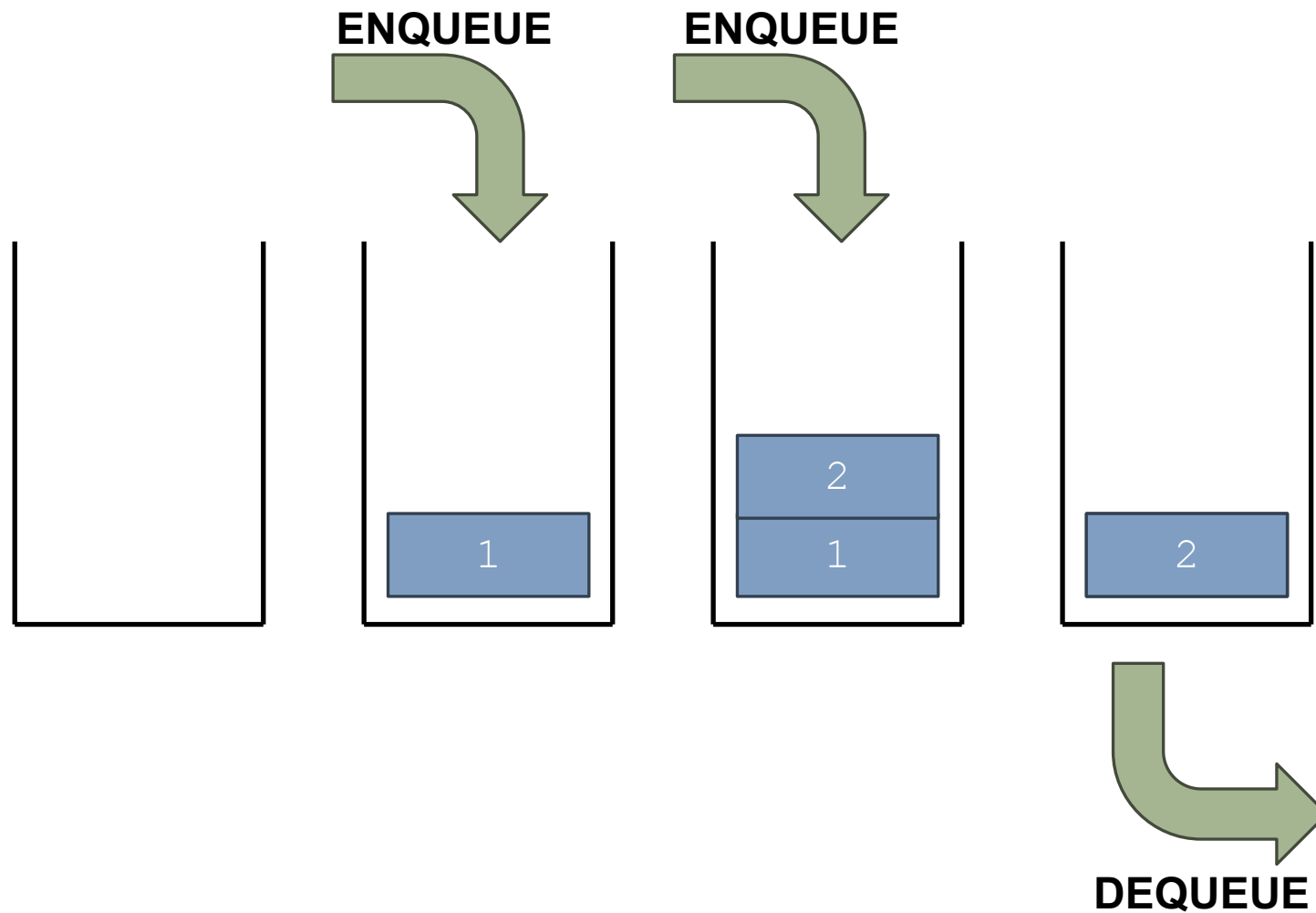
Coda (Queue): Esempio



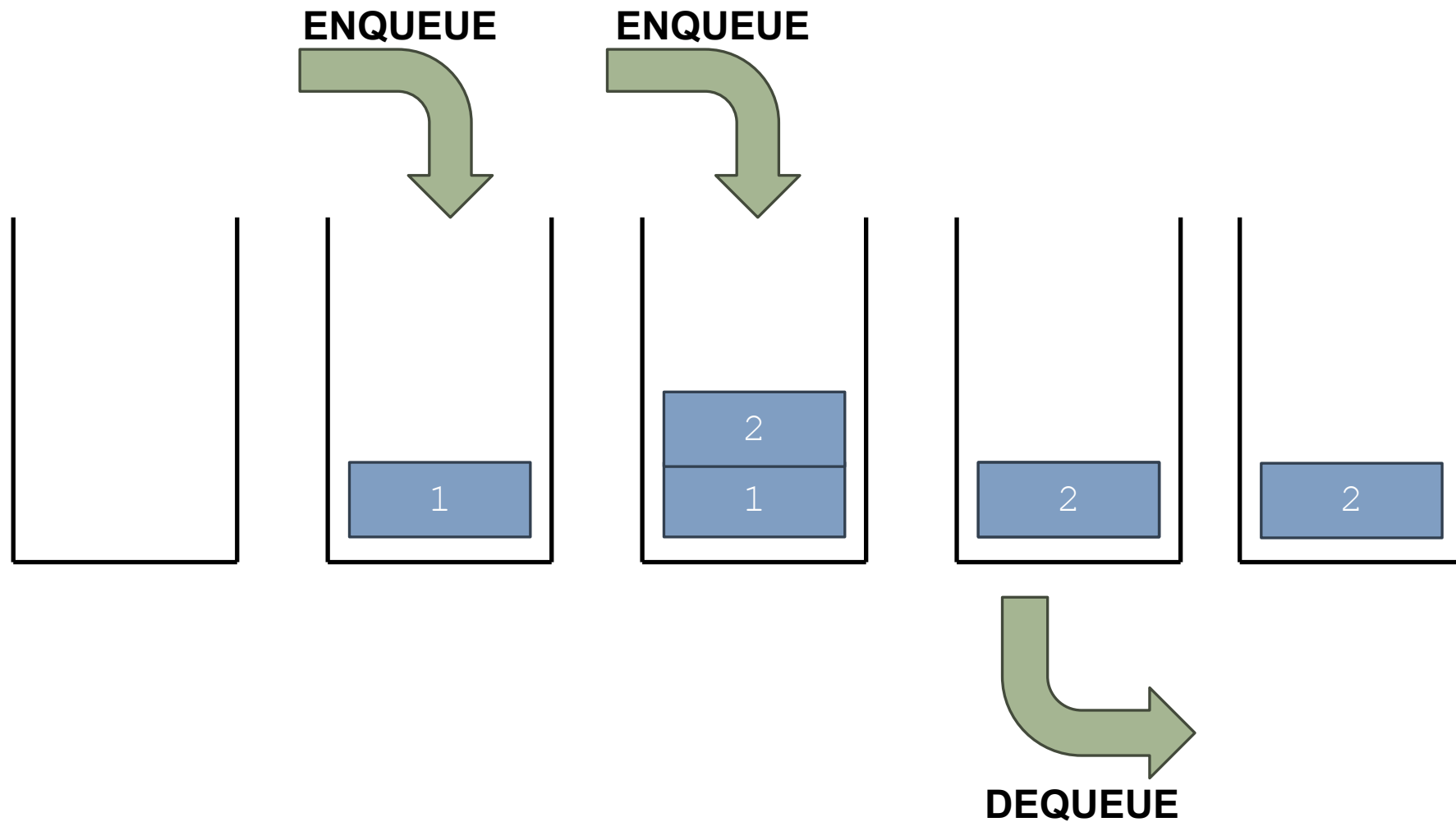
Coda (Queue): Esempio



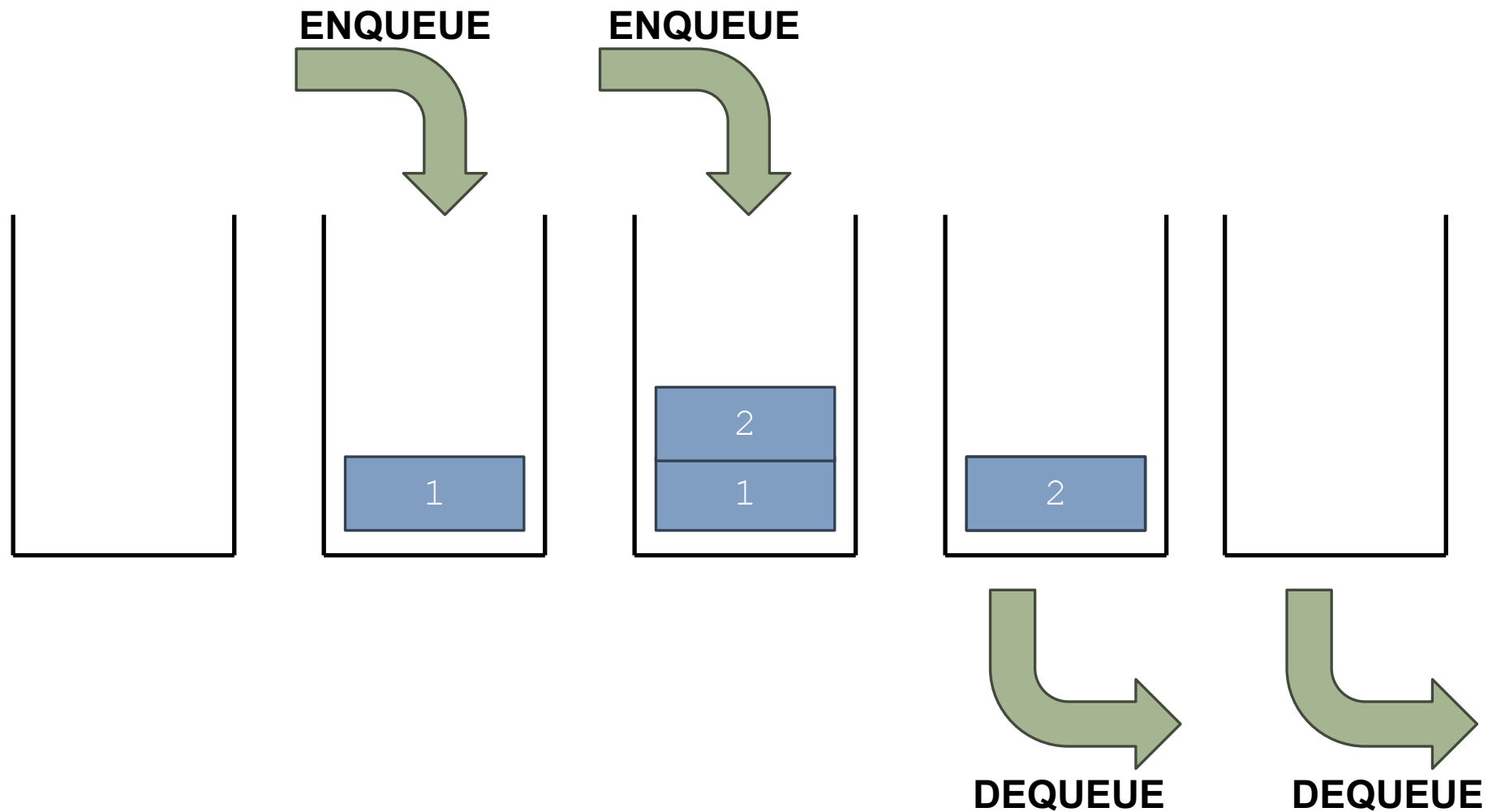
Coda (Queue): Esempio



Coda (Queue): Esempio



Coda (Queue): Esempio



Queue (Coda) in C I



1.

•

2.

•

•

•

•

•

•

•

•

Queue (Coda) in C I



1. Nome del tipo

- Queue (di Element)

2.

- -
-
-
-
-
-
-

Queue (Coda) in C I

1. Nome del tipo

- `Queue (di Element)`

2. Signature e semantica (astratta) delle operazioni fondamentali:

- `Queue mkQueue()`
 - Restituisce una coda vuota
- `void enqueue(Queue, const Element)`
 - Aggiunge un elemento in fondo alla coda
- `Element dequeue(Queue)`
 - Toglie e restituisce l'elemento in testa alla coda
- `_Bool isEmptyStack(const Stack)`
 - Controlla se la coda è vuota

Queue (Coda) in C II



2. ... altre operazioni, non sempre presenti sono:

- `Element front(const Queue)`
 - Restituisce l'elemento in testa alla coda (senza toglierlo)
- `Element rear(const Queue)`
 - Restituisce l'elemento in fondo alla coda (senza toglierlo)
- `Element peek(const Queue, int pos)`
 - Restituisce l'elemento nella posizione data (senza toglierlo)
- `int queueSize(const Queue)`
 - Restituisce il numero degli elementi presenti nella coda
- `_Bool isFullQueue(const Queue)`
 - Controlla se la coda è piena
- `void dsStack(Queue*)`
 - Distrugge la coda, recuperando la memoria

Queue (Coda) in C III

2. ... varianti di `mkQueue`, `push` e `pop`:

- `Queue mkQueue(int)`
 - Crea una coda vuota di capacità (fissa o variabile) specificata
- `_Bool enqueue(Queue, const Element)`
 - Aggiunge un elemento in fondo alla coda e restituisce l'esito dell'operazione

3.

-
-

Queue (Coda) in C III



2. ... varianti di `mkQueue`, `push` e `pop`:

- `Queue mkQueue(int)`
 - Crea una coda vuota di capacità (fissa o variabile) specificata
- `_Bool enqueue(Queue, const Element)`
 - Aggiunge un elemento in fondo alla coda e restituisce l'esito dell'operazione

3.

-
-

Perché può fallire?

Queue (Coda) in C III



2. ... varianti di `mkQueue`, `push` e `pop`:

- `Queue mkQueue(int)`
 - Crea una coda vuota di capacità (fissa o variabile) specificata
- `_Bool enqueue(Queue, const Element)`
 - Aggiunge un elemento in fondo alla coda e restituisce l'esito dell'operazione

3. Implementazioni, ne vedremo due:

-
-

Perché può fallire?

Queue (Coda) in C III



2. ... varianti di `mkQueue`, `push` e `pop`:

- `Queue mkQueue(int)`
 - Crea una coda vuota di capacità (fissa o variabile) specificata
- `_Bool enqueue(Queue, const Element)`
 - Aggiunge un elemento in fondo alla coda e restituisce l'esito dell'operazione

3. Implementazioni, ne vedremo due:

- Tramite lista
-

Perché può fallire?

Queue (Coda) in C III



2. ... varianti di `mkQueue`, `push` e `pop`:

- `Queue mkQueue(int)`
 - Crea una coda vuota di capacità (fissa o variabile) specificata
- `_Bool enqueue(Queue, const Element)`
 - Aggiunge un elemento in fondo alla coda e restituisce l'esito dell'operazione

3. Implementazioni, ne vedremo due:

- Tramite lista
- Tramite array

Perché può fallire?

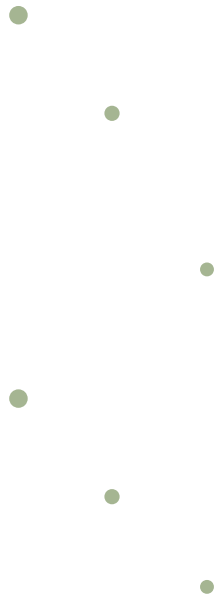
Esempio: Coda di char

- Nome del tipo:
 - `CharQueueADT`
- Consideriamo sette operazioni:
 - `CharQueueADT mkStack()`
 - Restituisce una pila vuota
 - `void dsQueue(CharQueueADT *qs)`
 - Distrugge la pila
 - `Bool enqueue(CharQueueADT q, char e)`
 - Aggiunge un `char` alla pila
 - `char dequeue(CharQueueADT q)`
 - Toglie e restituisce il `char` in testa alla coda
 - `_Bool isEmptyQueue(const CharQueueADT q)`
 - Controlla se la coda è vuota
 - `int queueSize(const CharQueueADT q)`
 - Restituisce il numero di elementi nella pila
 - `char peek(CharQueueADT q, int position)`
 - Restituisce l'elemento in posizione (senza toglierlo)
- Tipo e signature definite in `charQueueADT.h`

test_charQueueADT.c



- Come procediamo?



test_charQueueADT.c



- Come procediamo?
- In modo iterativo e incrementale applicando la metodologia TDD
 -
 -
 -
 -
 -

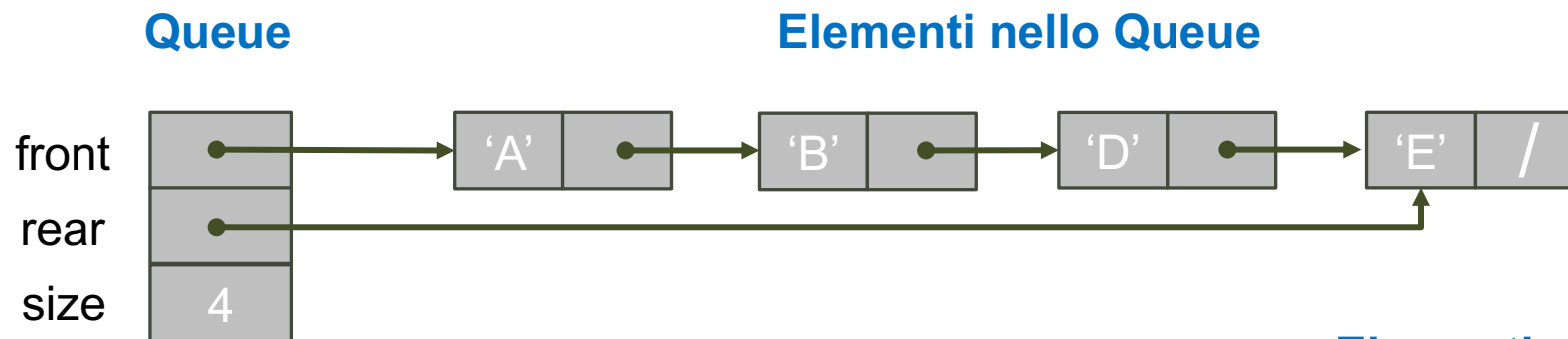
test_charQueueADT.c

- Come procediamo?
- In modo iterativo e incrementale applicando la metodologia TDD
 - Sviluppiamo (due) implementazioni per la specifica `charQueueADT.h` partendo da una test suite `test_charQueueADT.c`
 - Test suite omessa nei lucidi e lasciata come esercizio a voi
-
-
-

test_charQueueADT.c

- Come procediamo?
- In modo iterativo e incrementale applicando la metodologia TDD
 - Sviluppiamo (due) implementazioni per la specifica `charQueueADT.h` partendo da una test suite `test_charQueueADT.c`
 - Test suite omessa nei lucidi e lasciata come esercizio a voi
- Abbozziamo solo le funzioni principali
 - Esercizio:
 - Completare le implementazioni

Coda di char con Liste



- Implementazione con liste linkate:
 - Usa lista double-ended
 - Evita visita della lista
- Operazione enqueue:
 - Inserzione in coda: necessita solo di `rear`
- Operazione dequeue:
 - Rimozione dalla testa: necessita solo di `front`

Elementi nella Queue, i.e., Lista

```
typedef struct listNode ListNode,
    *ListNodePtr;

struct listNode {
    char data;
    ListNodePtr next;
};
```

Queue

```
struct charQueue {
    ListNodePtr front, rear;
    int size;
};
```


Coda di char con Liste: mkQueue ()

- Crea una queue
 - Con lista vuota
 - front e rear entrambi NULL
 - E 0 elementi

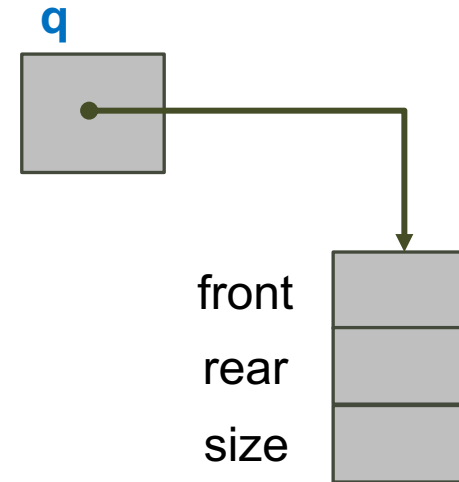
```
CharQueueADT mkQueue () {
```

```
}
```

Coda di char con Liste: mkQueue ()

- Crea una queue
 - Con lista vuota
 - front e rear entrambi NULL
 - E 0 elementi

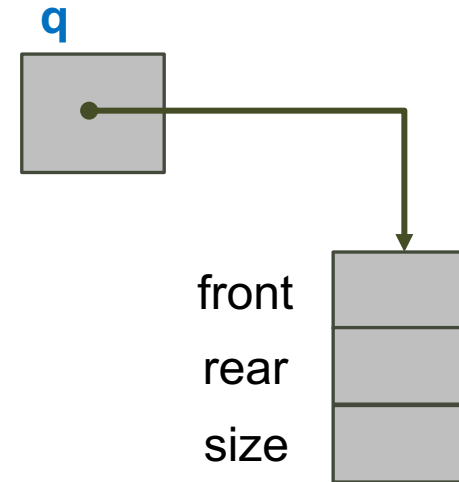
```
CharQueueADT mkQueue() {  
    CharQueueADT q = malloc(sizeof(*q));  
  
  
  
  
  
  
  
  
  
}
```



Coda di char con Liste: mkQueue ()

- Crea una queue
 - Con lista vuota
 - front e rear entrambi NULL
 - E 0 elementi

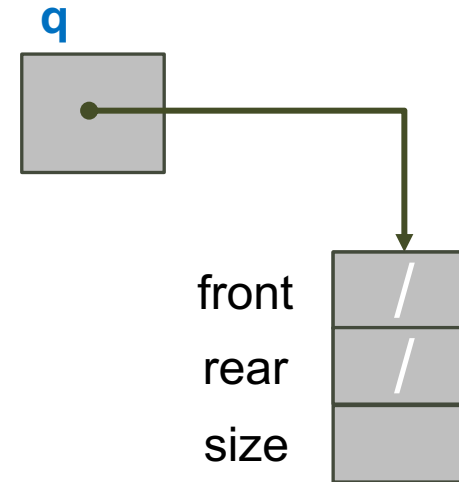
```
CharQueueADT mkQueue() {  
    CharQueueADT q = malloc(sizeof(*q));  
  
    // malloc() fallita?  
    if (q == NULL)  
        return NULL;  
  
}
```



Coda di char con Liste: mkQueue ()

- Crea una queue
 - Con lista vuota
 - front e rear entrambi NULL
 - E 0 elementi

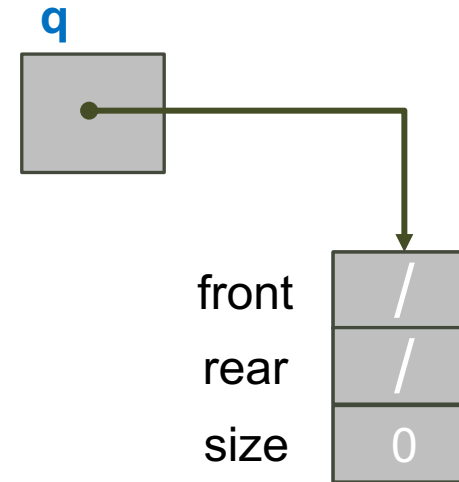
```
CharQueueADT mkQueue() {  
    CharQueueADT q = malloc(sizeof(*q));  
  
    // malloc() fallita?  
    if (q == NULL)  
        return NULL;  
  
    // lista vuota  
    q->front = q->rear = NULL;  
}
```



Coda di char con Liste: mkQueue ()

- Crea una queue
 - Con lista vuota
 - front e rear entrambi NULL
 - E 0 elementi

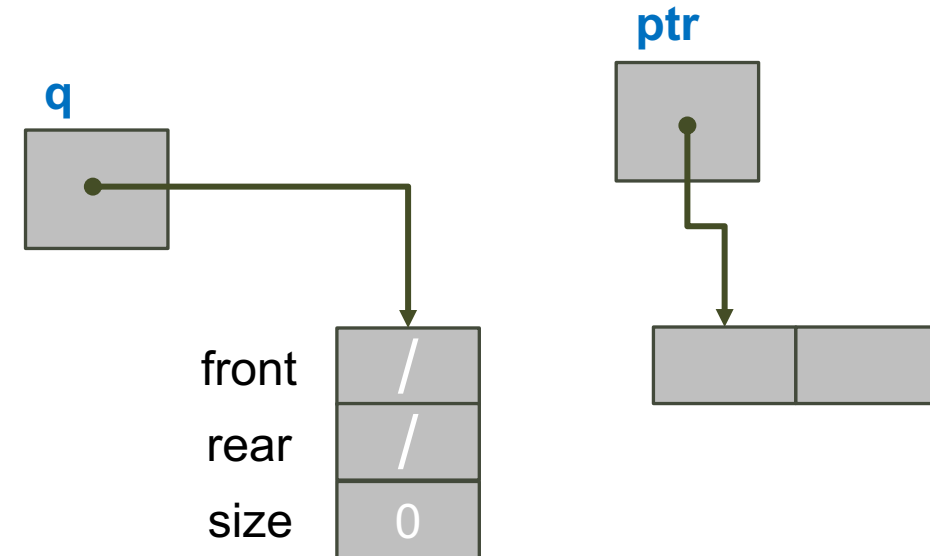
```
CharQueueADT mkQueue() {  
    CharQueueADT q = malloc(sizeof(*q));  
  
    // malloc() fallita?  
    if (q == NULL)  
        return NULL;  
  
    // lista vuota  
    q->front = q->rear = NULL;  
    q->size = 0;    // con 0 elementi  
    return q;  
}
```



Coda di char con Liste: enqueue ()

- Aggiunge un elemento in fondo alla coda
 - Se primo elemento: il fondo è anche testa

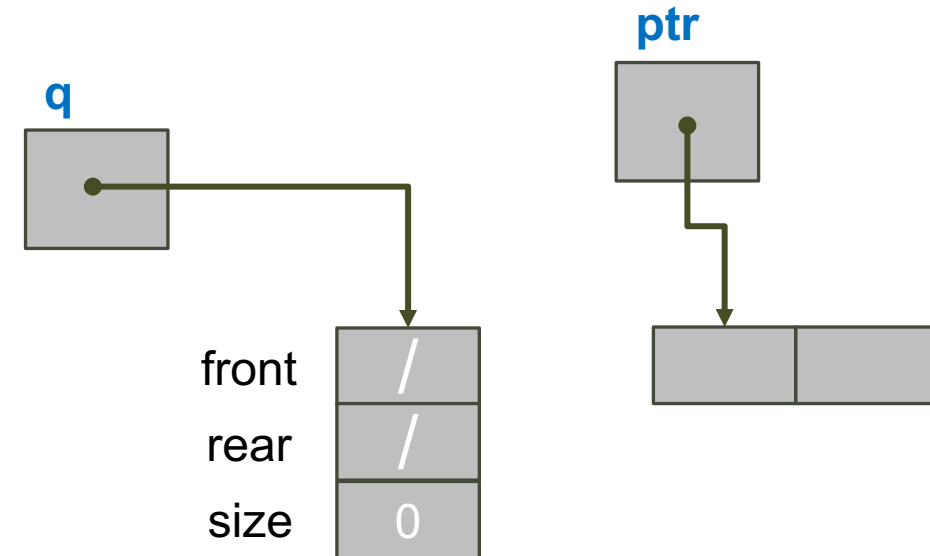
```
_Bool enqueue(CharQueueADT q, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
  
  
  
  
  
  
  
  
  
}
```



Coda di char con Liste: enqueue ()

- Aggiunge un elemento in fondo alla coda
 - Se primo elemento: il fondo è anche testa

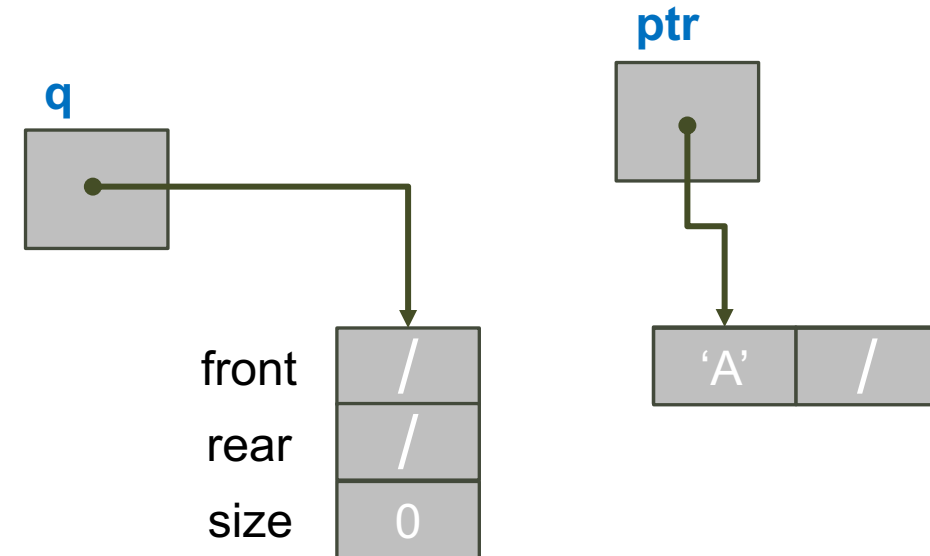
```
_Bool enqueue(CharQueueADT q, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL) return 0;  
  
    }  
}
```



Coda di char con Liste: enqueue ()

- Aggiunge un elemento in fondo alla coda
 - Se primo elemento: il fondo è anche testa

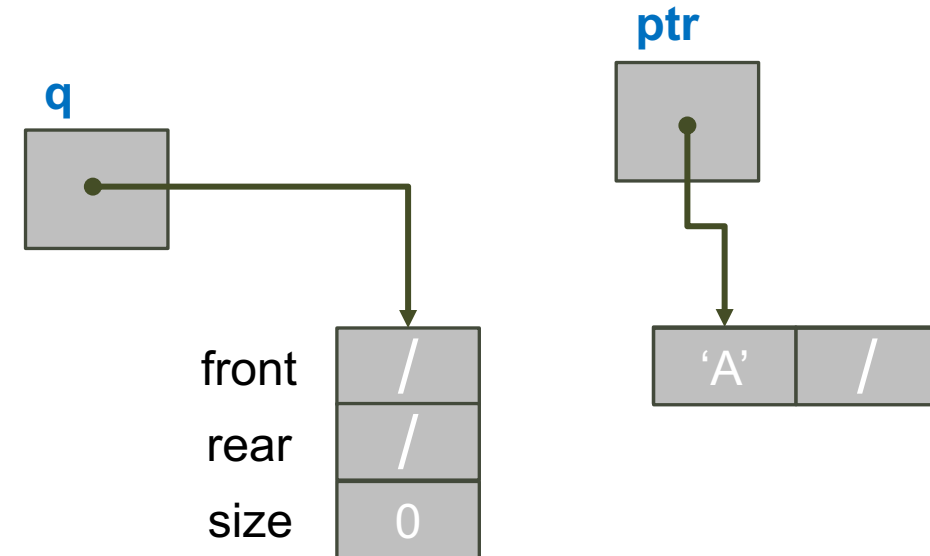
```
_Bool enqueue(CharQueueADT q, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL) return 0;  
    // Salva dato  
    ptr->data = e; ptr->next = NULL;  
  
}
```



Coda di char con Liste: enqueue ()

- Aggiunge un elemento in fondo alla coda
 - Se primo elemento: il fondo è anche testa

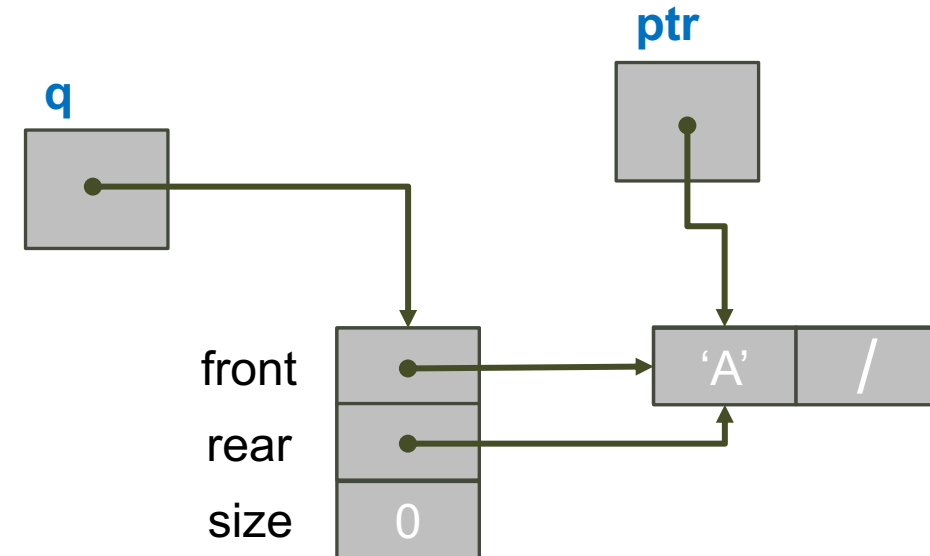
```
_Bool enqueue(CharQueueADT q, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL) return 0;  
    // Salva dato  
    ptr->data = e; ptr->next = NULL;  
  
    // Primo elemento?  
    if (isEmptyQueue(q)) {  
  
    }  
}
```



Coda di char con Liste: enqueue ()

- Aggiunge un elemento in fondo alla coda
 - Se primo elemento: il fondo è anche testa

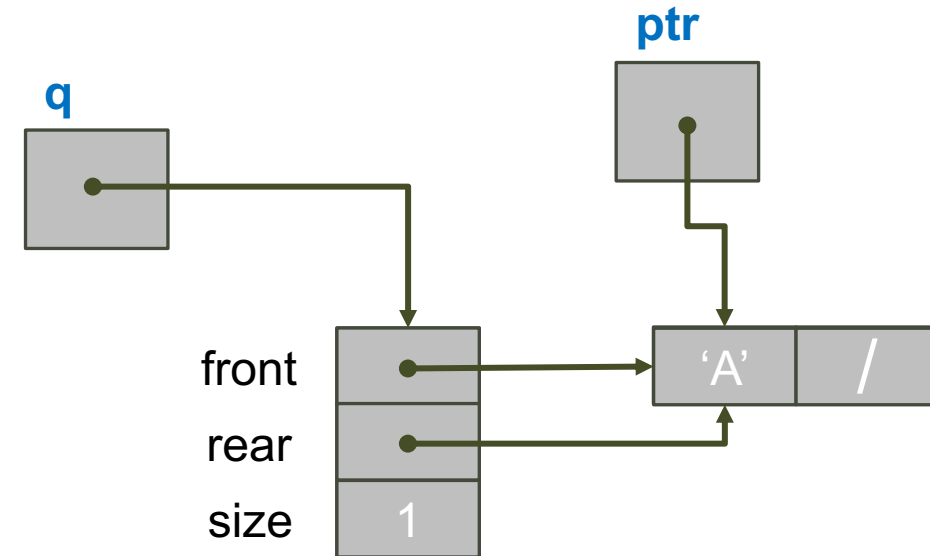
```
Bool enqueue(CharQueueADT q, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL) return 0;  
    // Salva dato  
    ptr->data = e; ptr->next = NULL;  
  
    // Primo elemento?  
    if(isEmptyQueue(q)) {  
        q->front = q->rear = ptr;  
    }  
}
```



Coda di char con Liste: enqueue ()

- Aggiunge un elemento in fondo alla coda
 - Se primo elemento: il fondo è anche testa

```
_Bool enqueue(CharQueueADT q, char e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL) return 0;  
    // Salva dato  
    ptr->data = e; ptr->next = NULL;  
  
    // Primo elemento?  
    if(isEmptyQueue(q)) {  
        q->front = q->rear = ptr;  
    } else {  
        // Inserisci in coda (da completare)  
    }  
    q->size++;  
    return 1;  
}
```

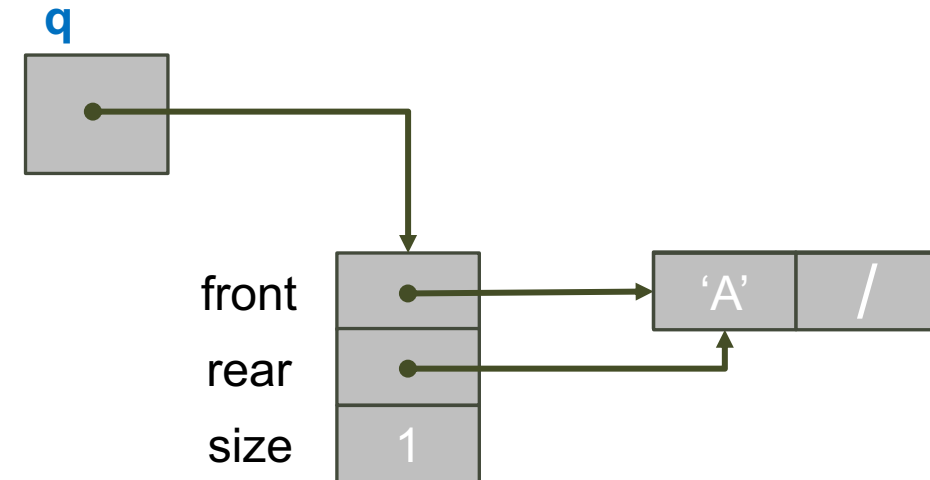


Coda di char con Liste: dequeue ()

- Rimuovi un elemento in testa alla coda
 - Se ultimo elemento: la testa è anche il fondo

```
char dequeue(CharQueueADT q) {
```

```
}
```

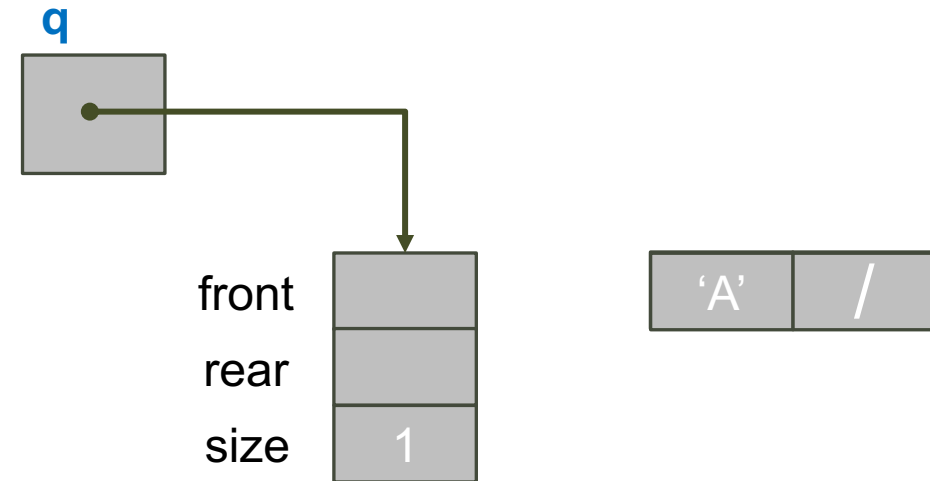


Coda di char con Liste: dequeue ()

- Rimuovi un elemento in testa alla coda
 - Se ultimo elemento: la testa è anche il fondo

```
char dequeue(CharQueueADT q) {
    // queue vuota?
    if (isEmptyQueue(q)) {exit(EXIT_FAILURE);}

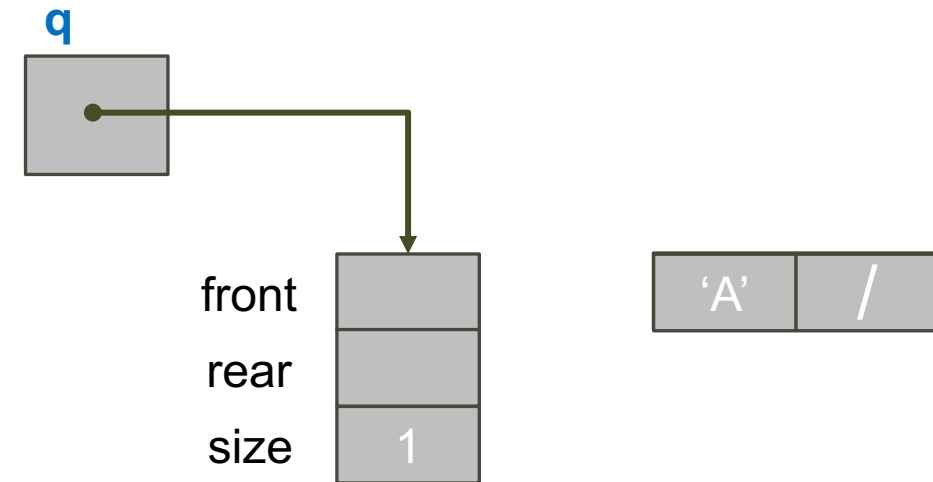
}
```



Coda di char con Liste: dequeue ()

- Rimuovi un elemento in testa alla coda
 - Se ultimo elemento: la testa è anche il fondo

```
char dequeue(CharQueueADT q) {  
    // queue vuota?  
    if(isEmptyQueue(q)) {exit(EXIT_FAILURE);}   
  
    char e = q->front->data;  
  
}
```



Coda di char con Liste: dequeue ()

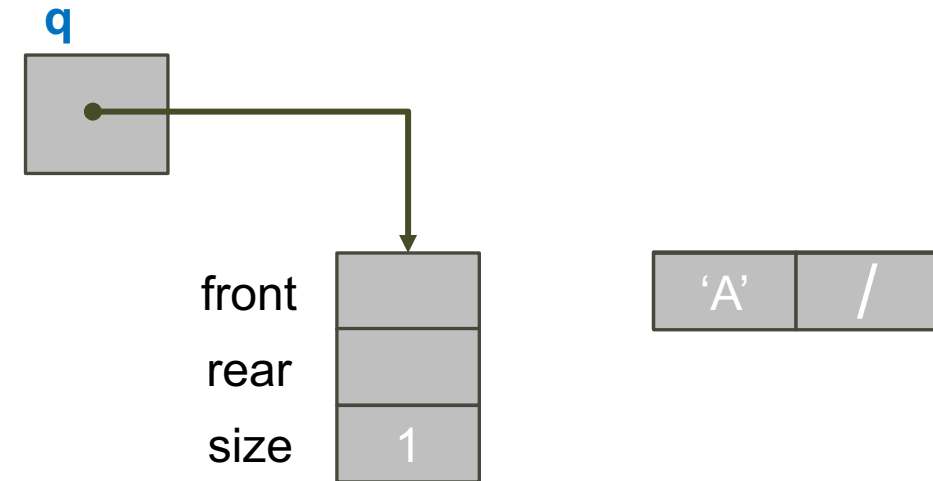
- Rimuovi un elemento in testa alla coda
 - Se ultimo elemento: la testa è anche il fondo

```
char dequeue(CharQueueADT q) {
    // queue vuota?
    if(isEmptyQueue(q)) {exit(EXIT_FAILURE);}

    char e = q->front->data;

    // Ultimo elemento?
    if(sizeQueue(q) == 1) {

    }
}
```



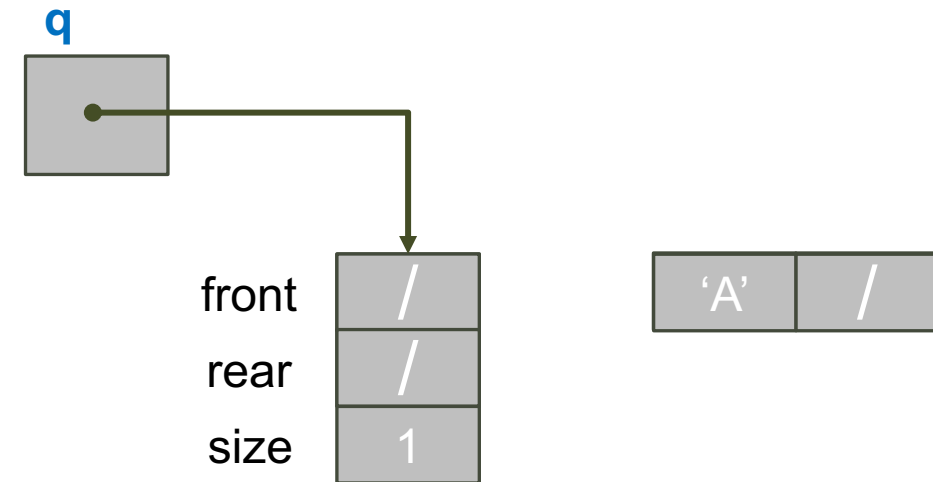
Coda di char con Liste: dequeue ()

- Rimuovi un elemento in testa alla coda
 - Se ultimo elemento: la testa è anche il fondo

```
char dequeue(CharQueueADT q) {
    // queue vuota?
    if(isEmptyQueue(q)) {exit(EXIT_FAILURE);}

    char e = q->front->data;

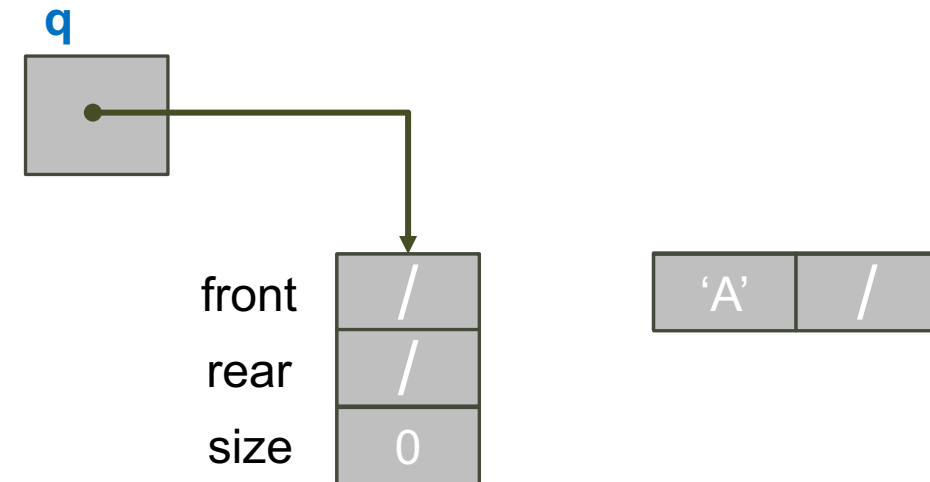
    // Ultimo elemento?
    if(sizeQueue(q) == 1) {
        q->front = q->rear = NULL;
    }
}
```



Coda di char con Liste: dequeue ()

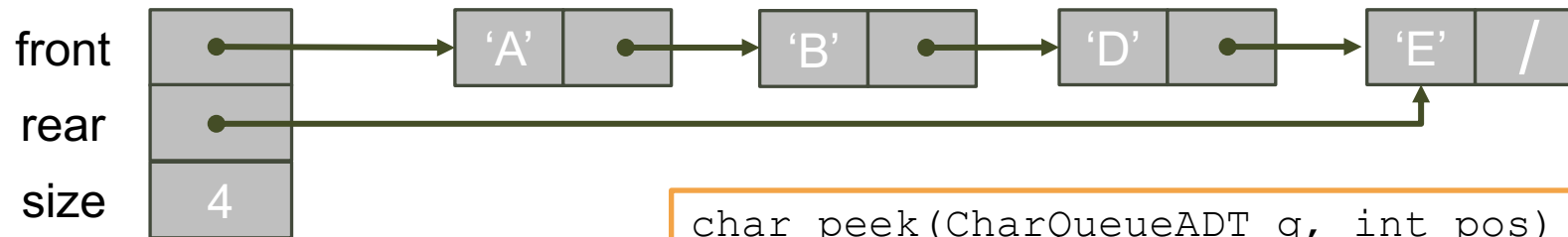
- Rimuovi un elemento in testa alla coda
 - Se ultimo elemento: la testa è anche il fondo

```
char dequeue(CharQueueADT q) {  
    // queue vuota?  
    if(isEmptyQueue(q)) {exit(EXIT_FAILURE);}   
  
    char e = q->front->data;  
  
    // Ultimo elemento?  
    if(sizeQueue(q) == 1) {  
        q->front = q->rear = NULL;  
    } else {  
        // Rimuovi dalla testa (da completare)  
    }  
    q->size--;  
    return e;  
}
```



Coda di char con Liste: peek ()

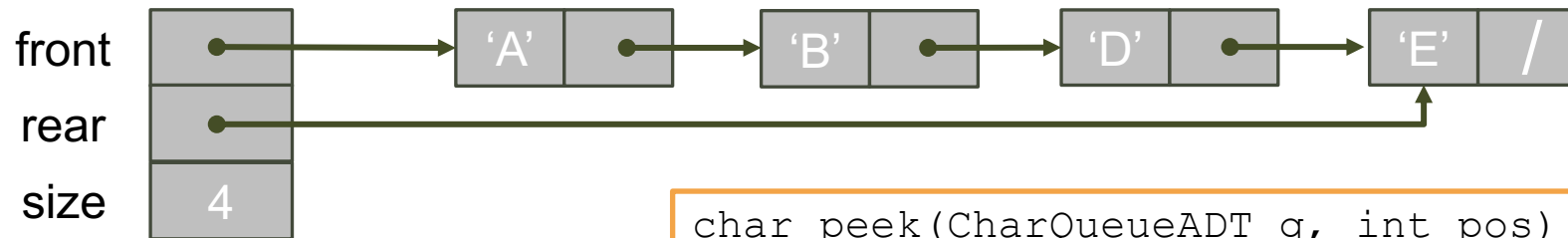
- Restituisce l'elemento in posizione `pos` (senza toglierlo)
 - Esempio: `pos == 2`



```
char peek(CharQueueADT q, int pos) {
```

Coda di char con Liste: peek ()

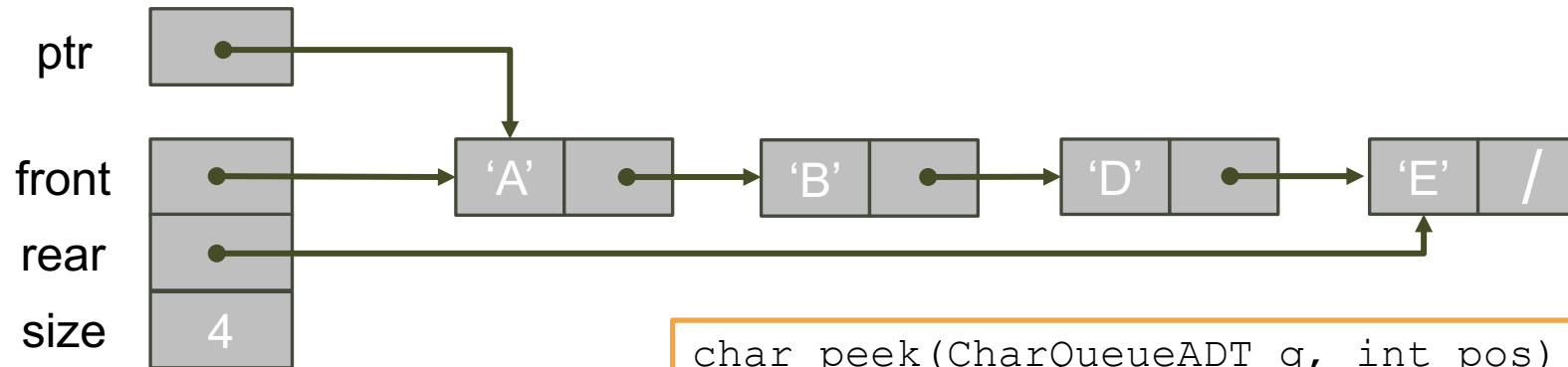
- Restituisce l'elemento in posizione `pos` (senza toglierlo)
 - Esempio: `pos == 2`



```
char peek(CharQueueADT q, int pos) {  
    // Visita la lista fino a pos  
  
}
```

Coda di char con Liste: peek ()

- Restituisce l'elemento in posizione `pos` (senza toglierlo)
 - Esempio: `pos == 2`

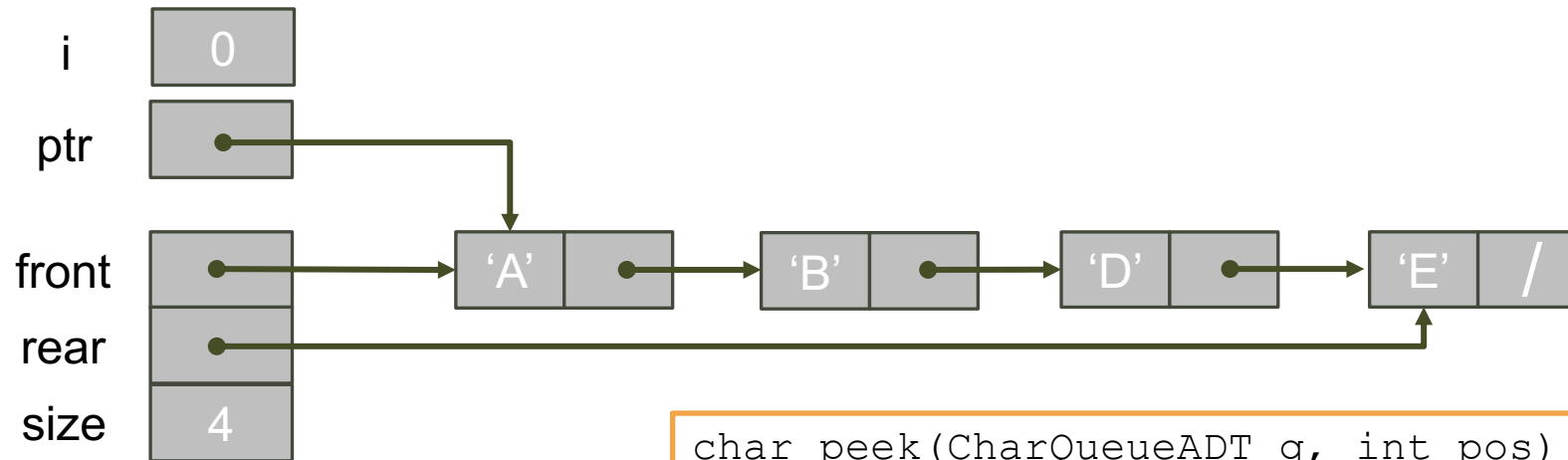


```
char peek(CharQueueADT q, int pos) {
    // Visita la lista fino a pos
    ListNodePtr ptr = q->front;

}
```

Coda di char con Liste: peek ()

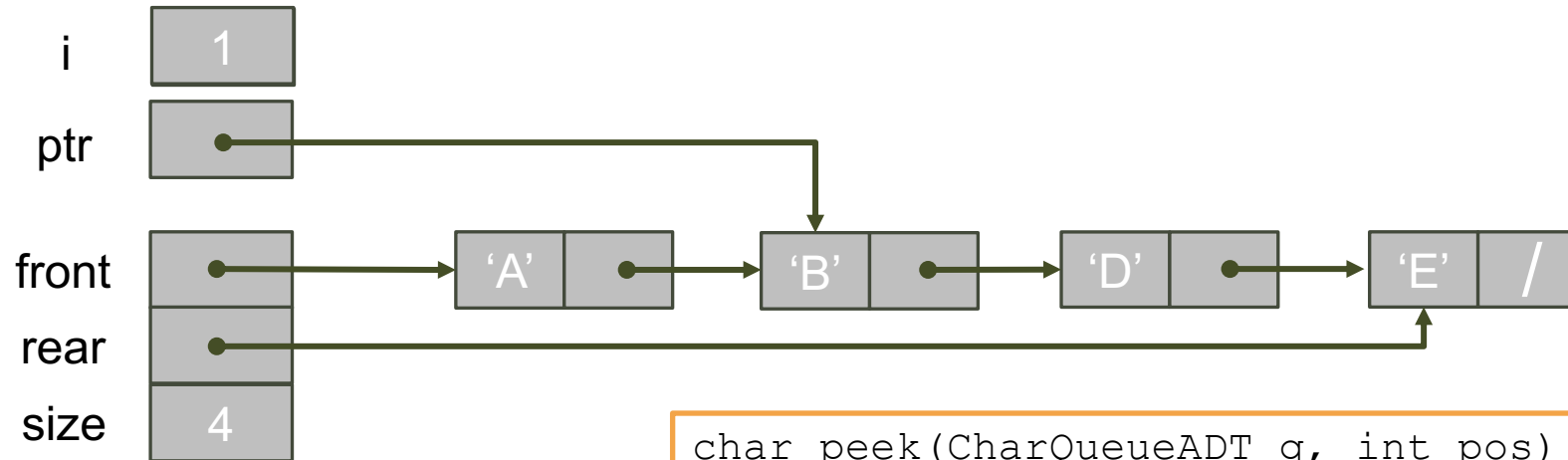
- Restituisce l'elemento in posizione `pos` (senza toglierlo)
 - Esempio: `pos == 2`



```
char peek(CharQueueADT q, int pos) {
    // Visita la lista fino a pos
    ListNodePtr ptr = q->front;
    for(int i=0; ptr != NULL && i < pos; i++) {
        ptr = ptr->next;
    }
}
```

Coda di char con Liste: peek ()

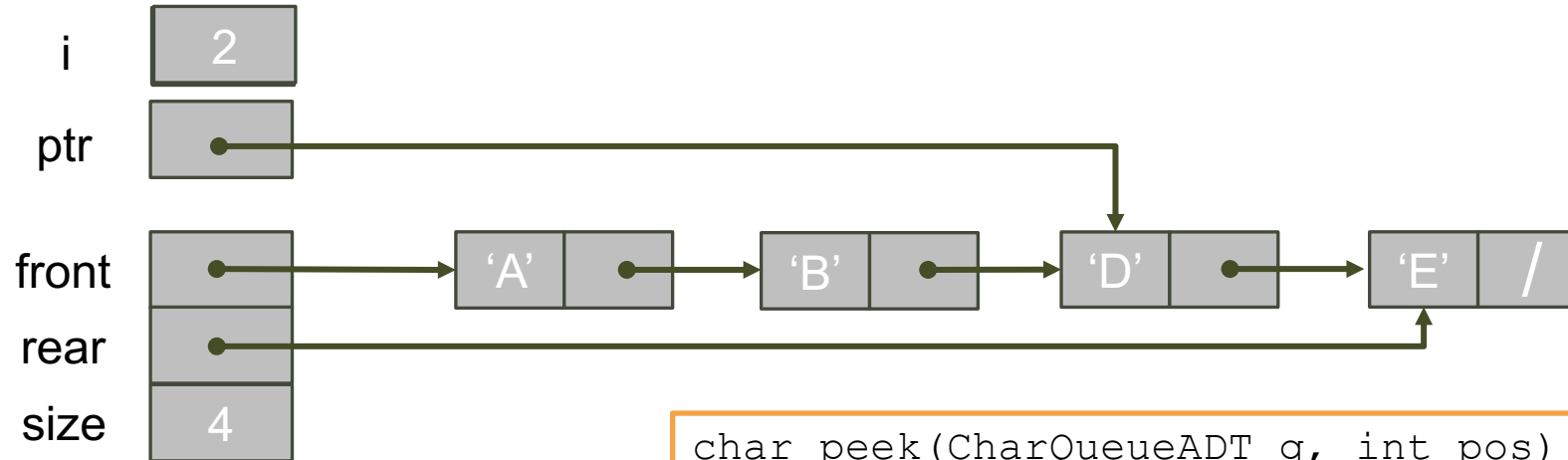
- Restituisce l'elemento in posizione `pos` (senza toglierlo)
 - Esempio: `pos == 2`



```
char peek(CharQueueADT q, int pos) {
    // Visita la lista fino a pos
    ListNodePtr ptr = q->front;
    for(int i=0; ptr != NULL && i < pos; i++) {
        ptr = ptr->next;
    }
}
```

Coda di char con Liste: peek ()

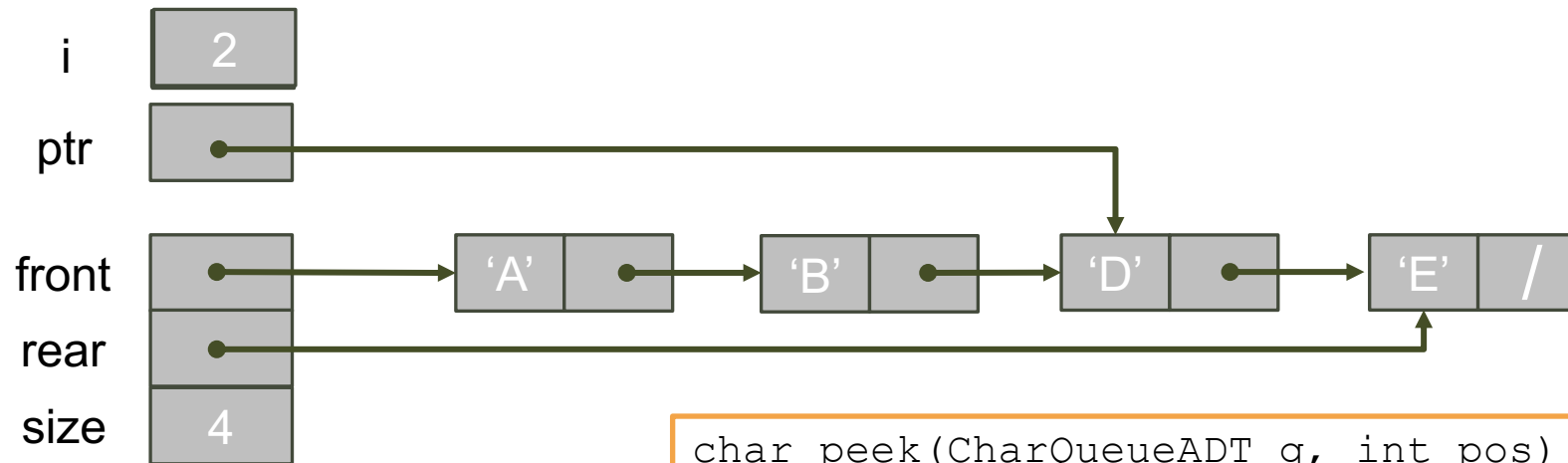
- Restituisce l'elemento in posizione `pos` (senza toglierlo)
 - Esempio: `pos == 2`



```
char peek(CharQueueADT q, int pos) {
    // Visita la lista fino a pos
    ListNodePtr ptr = q->front;
    for(int i=0; ptr != NULL && i < pos; i++) {
        ptr = ptr->next;
    }
}
```


Coda di char con Liste: peek ()

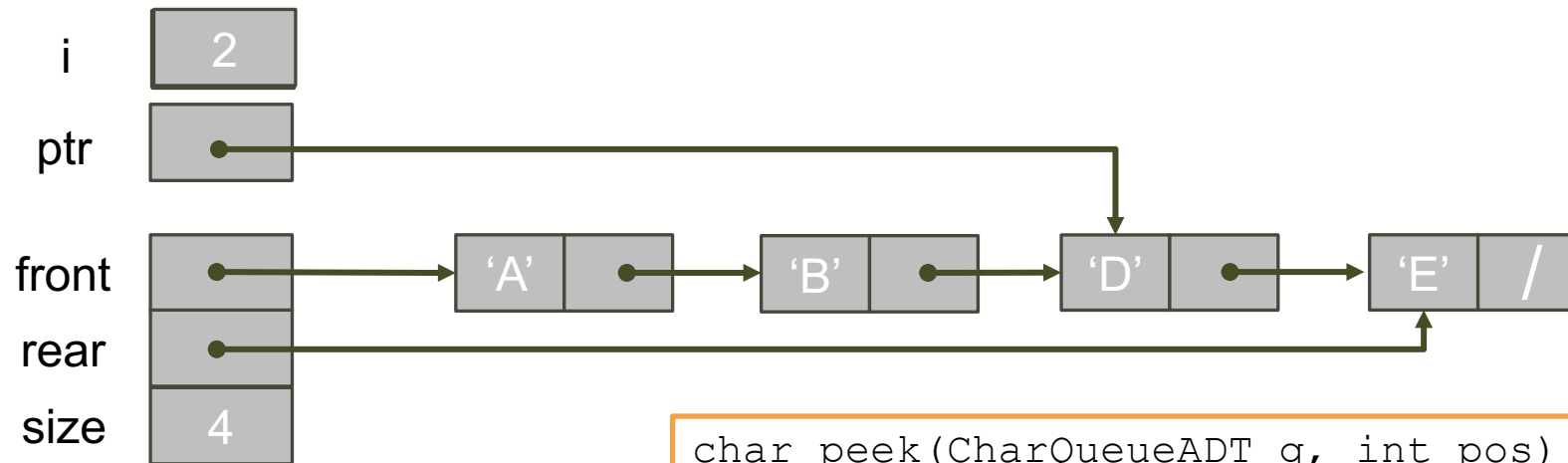
- Restituisce l'elemento in posizione `pos` (senza toglierlo)
 - Esempio: `pos == 2`



```
char peek(CharQueueADT q, int pos) {
    // Visita la lista fino a pos
    ListNodePtr ptr = q->front;
    for(int i=0; ptr != NULL && i < pos; i++) {
        ptr = ptr->next;
    }
    if(ptr == NULL) {exit(EXIT_FAILURE);} // se non esiste, fallisce
}
```

Coda di char con Liste: peek ()

- Restituisce l'elemento in posizione `pos` (senza toglierlo)
 - Esempio: `pos == 2`

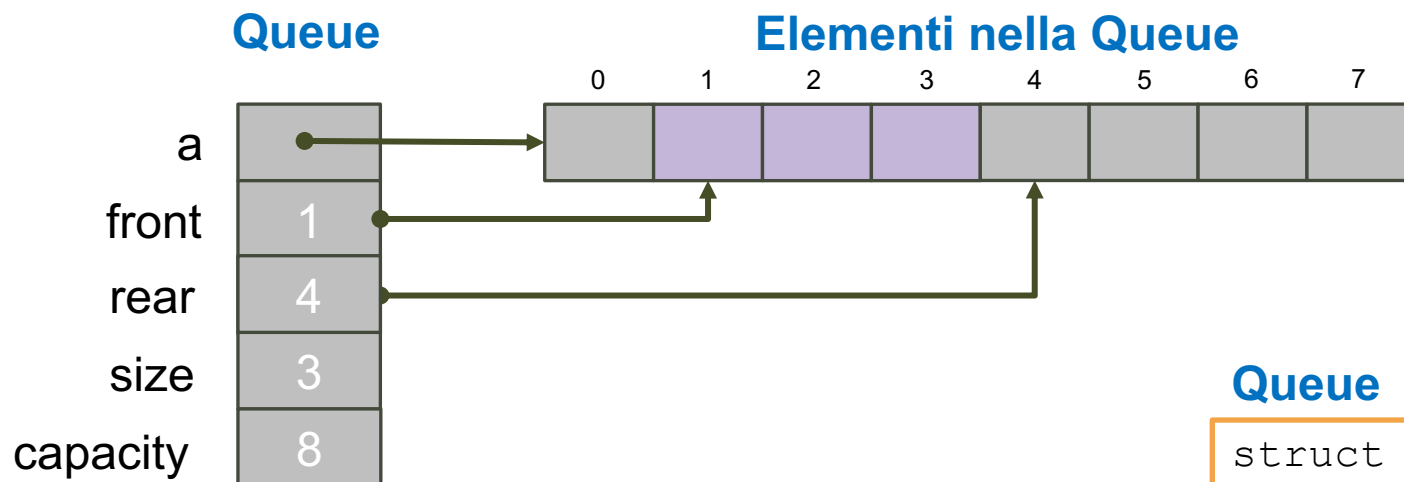


```
char peek(CharQueueADT q, int pos) {
    // Visita la lista fino a pos
    ListNodePtr ptr = q->front;
    for(int i=0; ptr != NULL && i < pos; i++) {
        ptr = ptr->next;
    }
    if(ptr == NULL) {exit(EXIT_FAILURE);} // se non esiste, fallisce
    return ptr->data;
}
```

Esercizio: Coda di `char` con Liste

- Completare:
 - `Bool enqueue(CharQueueADT q, char e)`
 - Aggiunge un `char` alla pila
 - `char dequeue(CharQueueADT q)`
 - Toglie e restituisce il `char` in testa alla coda
- ... e implementare le restanti operazioni:
 - `void dsQueue(CharQueueADT *qs)`
 - Distrugge la pila
 - `_Bool isEmptyQueue(const CharQueueADT q)`
 - Controlla se la coda è vuota
 - `int queueSize(const CharQueueADT q)`
 - Restituisce il numero di elementi nella pila
- Tipo e signature definite in `charQueueADT.h`

Coda di char con Array Dinamico

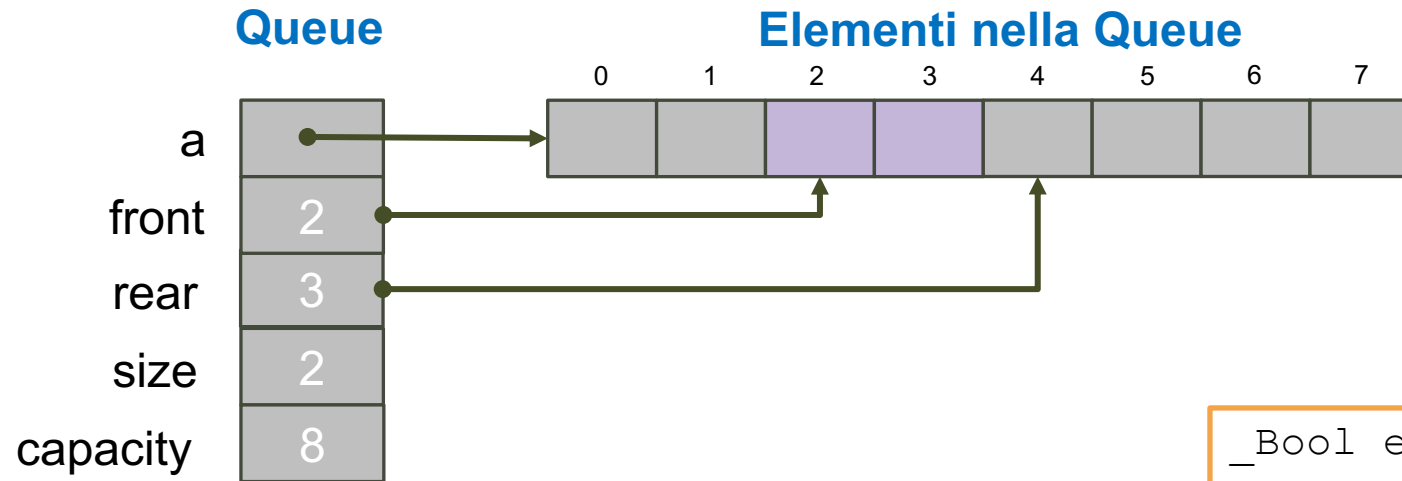


- Implementazione con array dinamico:
 - Elementi salvati nell'array
- Due indici che indicano:
 - Il primo elemento in coda (front)
 - La prima posizione libera (rear)

Queue

```
struct charQueue {  
    char* a; // memorizza gli elementi  
    int capacity; // dimensione dell'array.  
    int rear; // prima posizione libera in a  
    int front; /* posizione in a dove (se  
size > 0) è memorizzato il primo elemento  
della coda */  
    int size; // numero elementi nella coda  
};
```

Coda di char con Array: enqueue ()

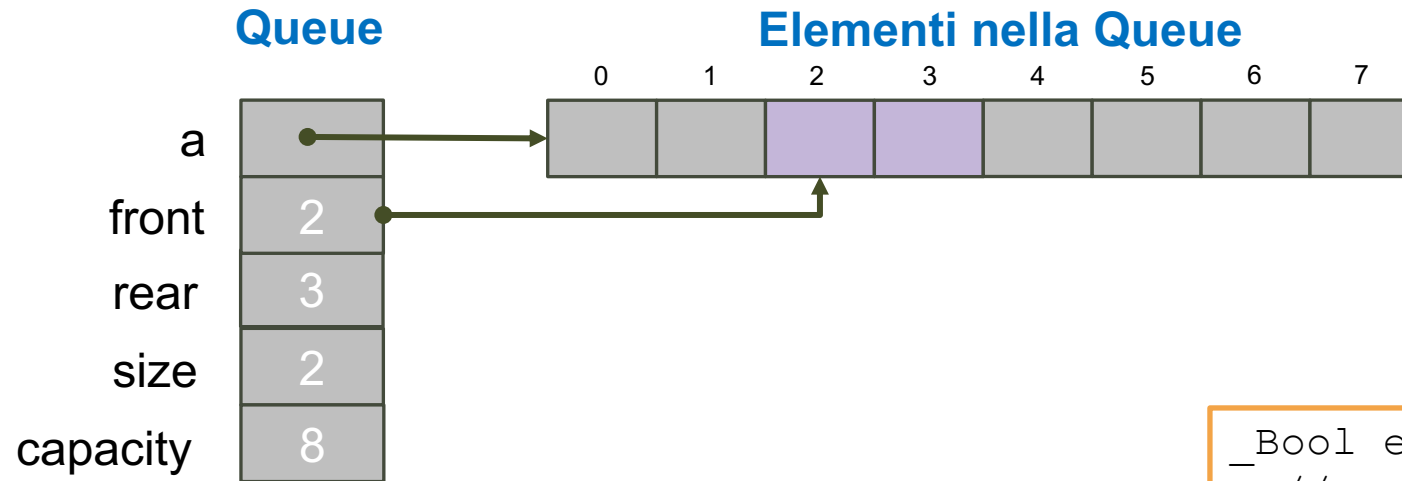


- Aggiunge un elemento in fondo alla coda

```
_Bool enqueue(CharQueueADT q, char e) {
```

```
}
```

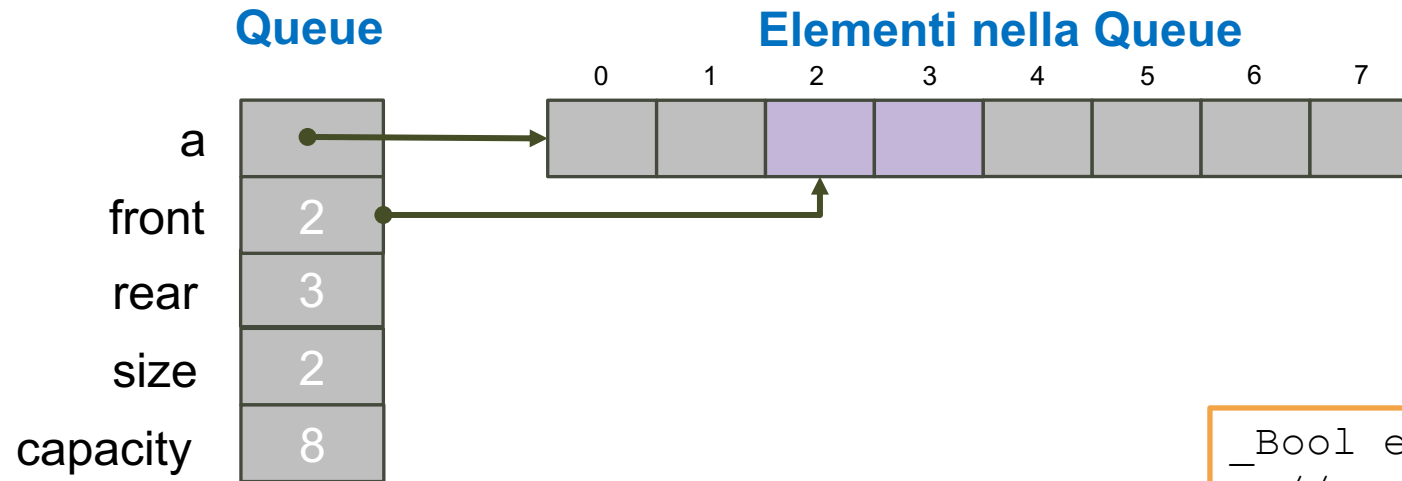
Coda di char con Array: enqueue ()



- Aggiunge un elemento in fondo alla coda

```
_Bool enqueue(CharQueueADT q, char e) {  
    // Manca spazio?  
    if(q->size >= q-capacity) {  
  
    }  
  
}
```

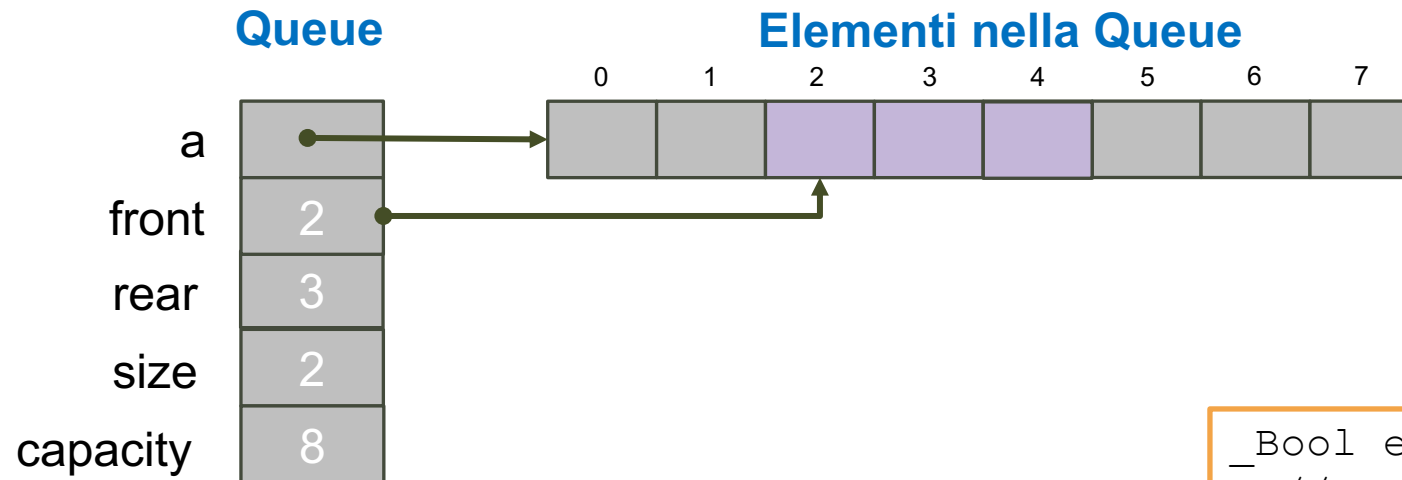
Coda di char con Array: enqueue ()



- Aggiunge un elemento in fondo alla coda

```
_Bool enqueue(CharQueueADT q, char e) {  
    // Manca spazio?  
    if(q->size >= q->capacity) {  
        /* Rialloca a con dimensione doppia (da  
        completare */  
    }  
}
```

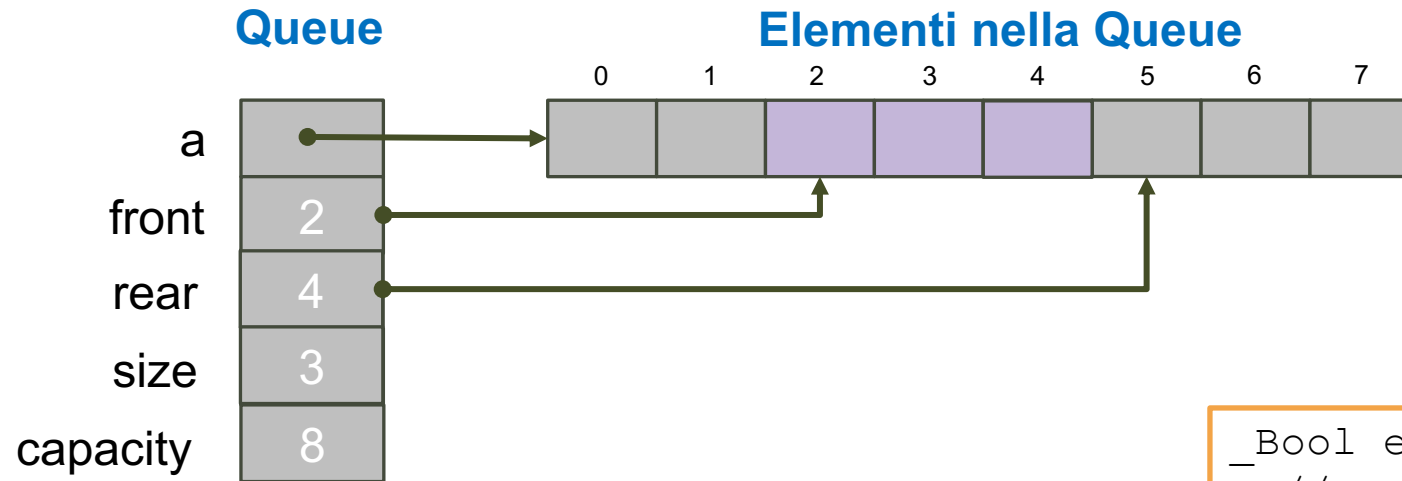
Coda di char con Array: enqueue ()



- Aggiunge un elemento in fondo alla coda

```
_Bool enqueue(CharQueueADT q, char e) {  
    // Manca spazio?  
    if(q->size >= q->capacity) {  
        /* Rialloca a con dimensione doppia (da  
        completare */  
    }  
    // Inserisci in posizione rear  
    q->a[q->rear] = e;  
}
```

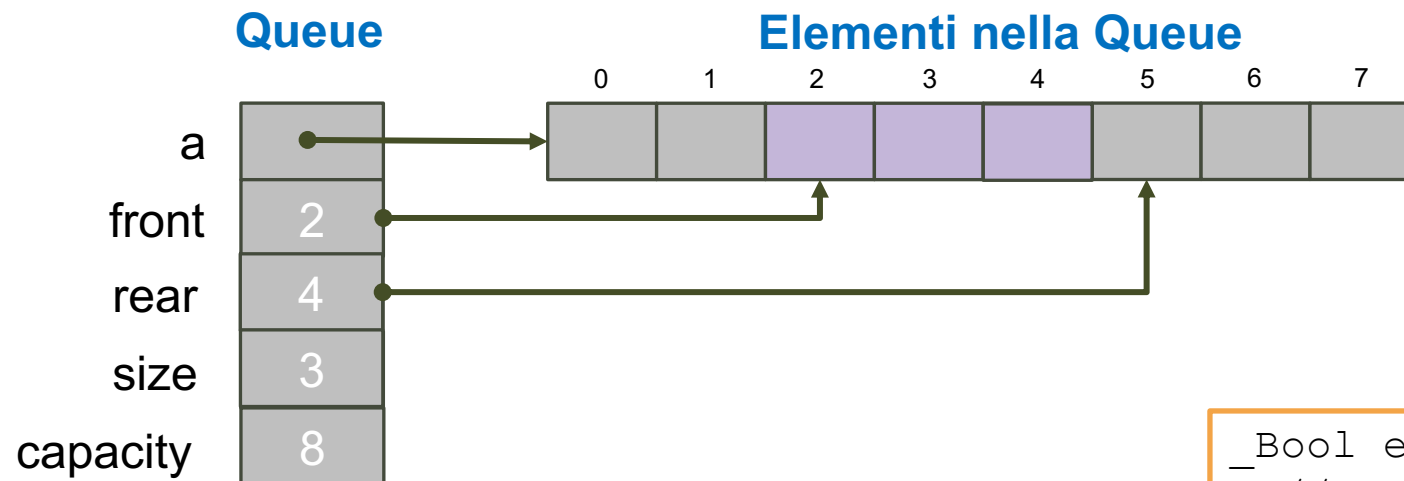

Coda di char con Array: enqueue ()



- Aggiunge un elemento in fondo alla coda

```
_Bool enqueue(CharQueueADT q, char e) {  
    // Manca spazio?  
    if(q->size >= q->capacity) {  
        /* Rialloca a con dimensione doppia (da  
        completare */  
    }  
    // Inserisci in posizione rear  
    q->a[q->rear] = e;  
    q->rear++;  
    q->size++;  
    return 1;  
}
```

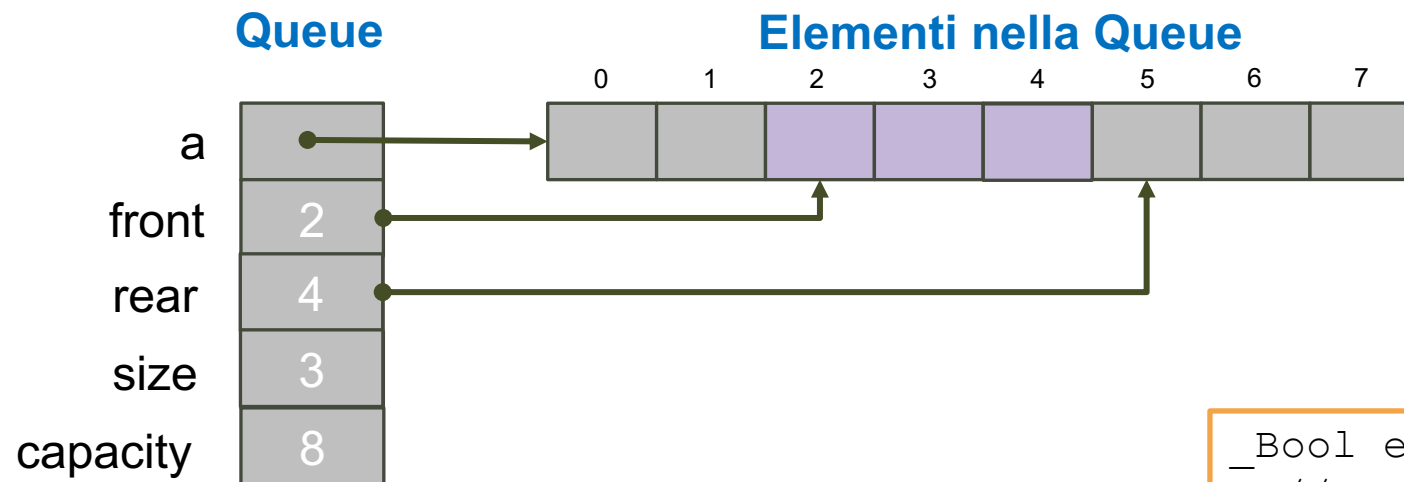
Coda di char con Array: enqueue ()



- Aggiunge un elemento in fondo alla coda
 - Cosa succede se arriviamo in fondo all'array?
 -
 -

```
_Bool enqueue(CharQueueADT q, char e) {  
    // Manca spazio?  
    if(q->size >= q->capacity) {  
        /* Rialloca a con dimensione doppia (da  
        completare */  
    }  
    // Inserisci in posizione rear  
    q->a[q->rear] = e;  
    q->rear++;  
    q->size++;  
    return 1;  
}
```

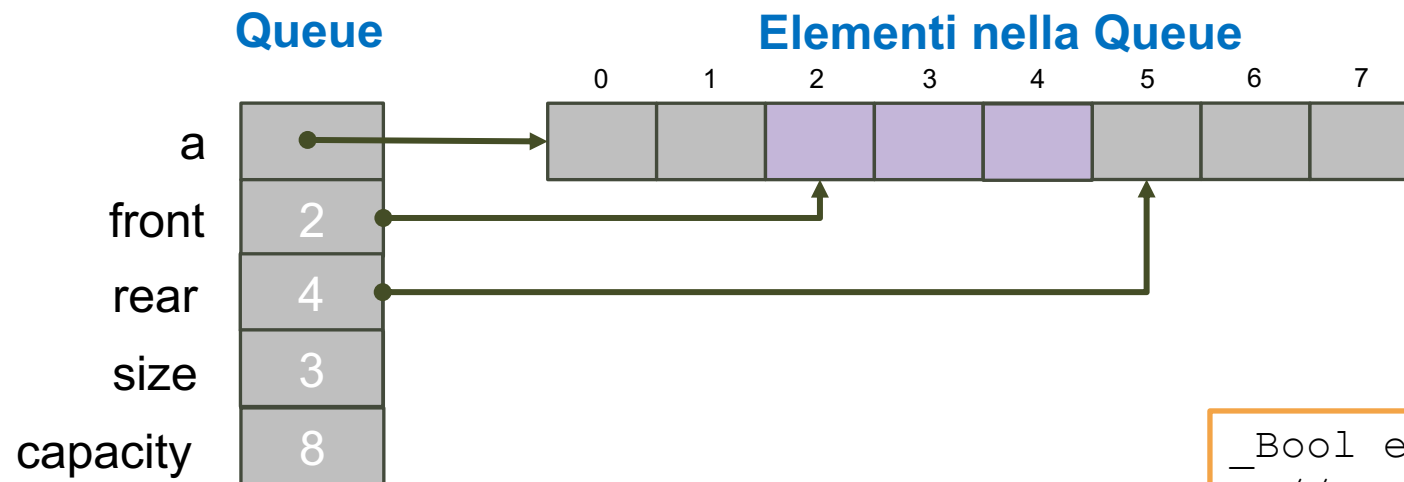
Coda di char con Array: enqueue ()



- Aggiunge un elemento in fondo alla coda
 - Cosa succede se arriviamo in fondo all'array?
 - Ri-inizia dal primo indice dell'array
 -

```
_Bool enqueue(CharQueueADT q, char e) {  
    // Manca spazio?  
    if(q->size >= q->capacity) {  
        /* Rialloca a con dimensione doppia (da  
        completare */  
    }  
    // Inserisci in posizione rear  
    q->a[q->rear] = e;  
    q->rear++;  
    q->size++;  
    return 1;  
}
```

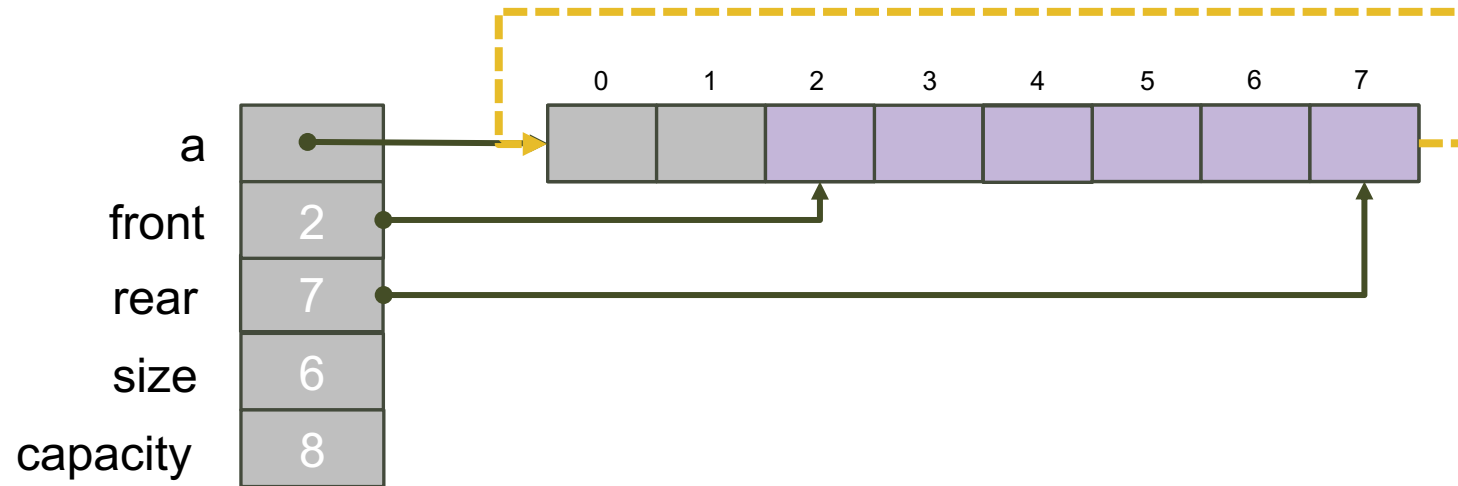
Coda di char con Array: enqueue ()



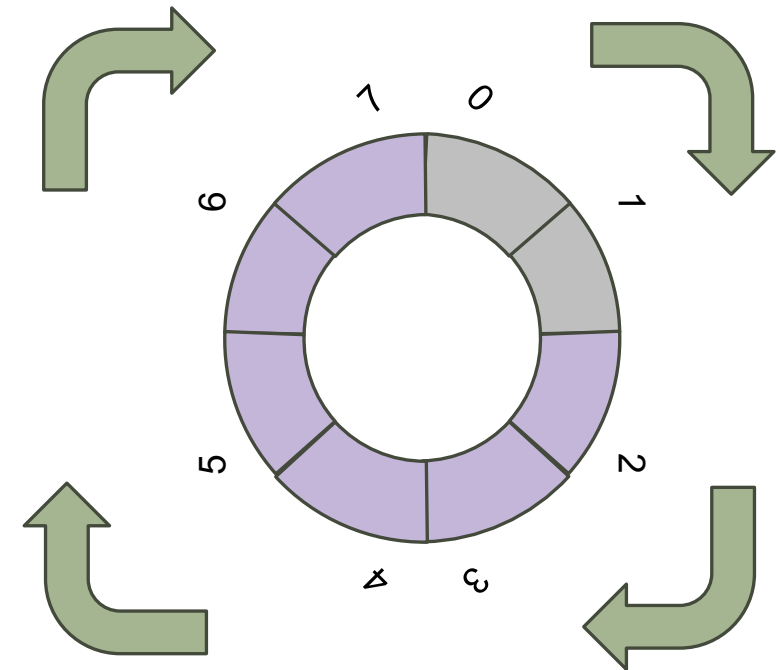
- Aggiunge un elemento in fondo alla coda
 - Cosa succede se arriviamo in fondo all'array?
 - Ri-inizia dal primo indice dell'array
 - Array circolare

```
_Bool enqueue(CharQueueADT q, char e) {  
    // Manca spazio?  
    if(q->size >= q->capacity) {  
        /* Rialloca a con dimensione doppia (da  
        completare */  
    }  
    // Inserisci in posizione rear  
    q->a[q->rear] = e;  
    q->rear++;  
    q->size++;  
    return 1;  
}
```

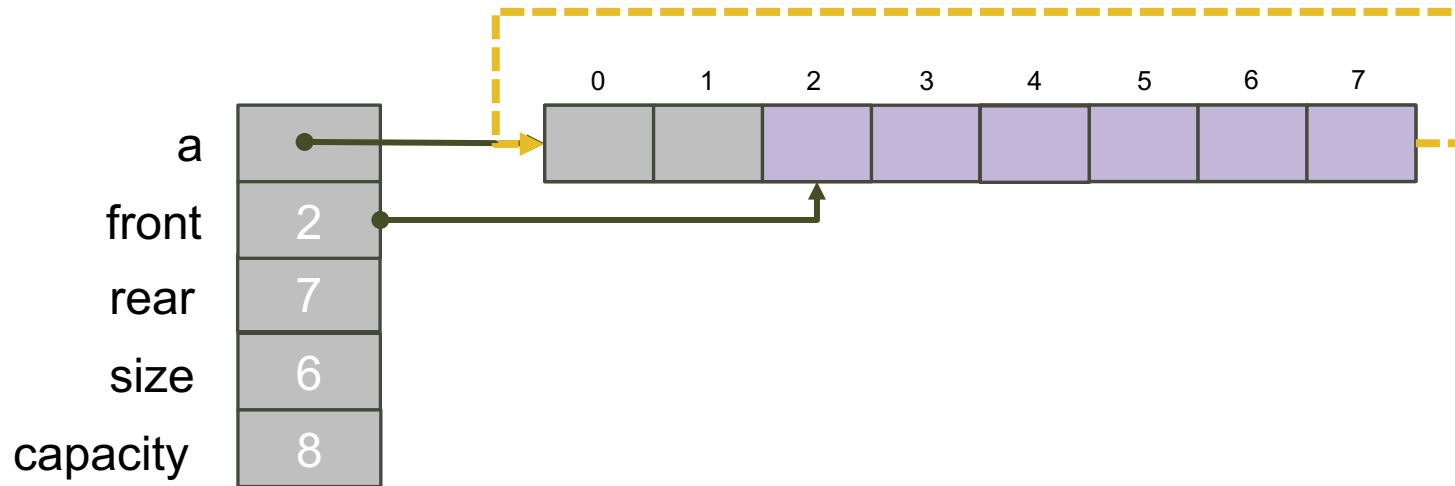
Coda Circolare



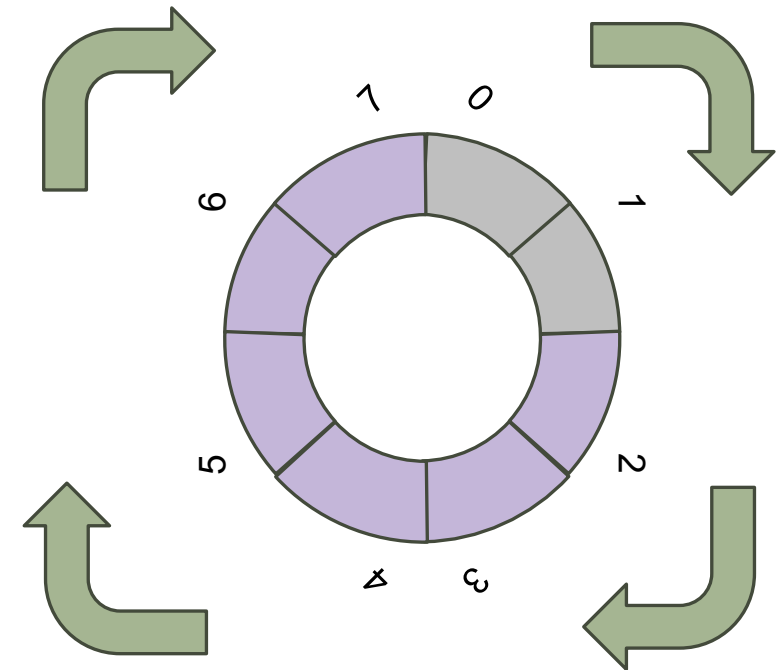
- Coda circolare:
 - Raggiunto `capacity`
 - Gli indici saltano allo 0
 -
 -



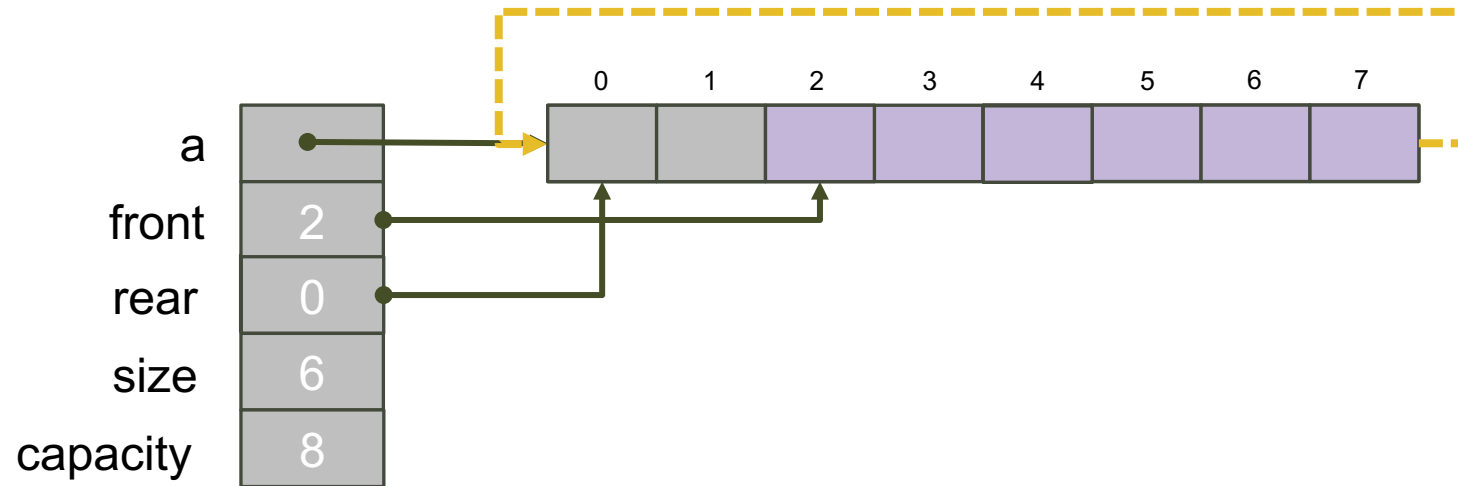
Coda Circolare



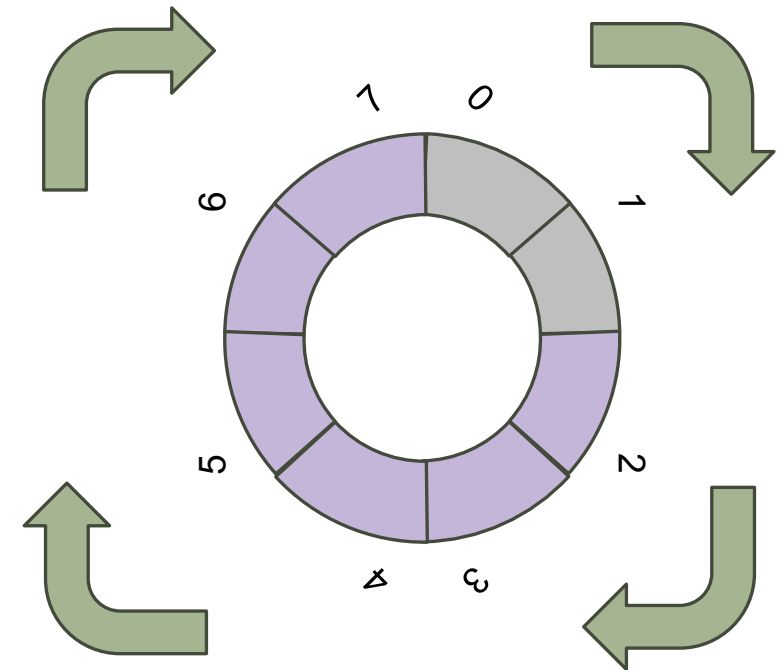
- Coda circolare:
 - Raggiunto `capacity`
 - Gli indici saltano allo 0
 - Esempio: `enqueue()`
 -



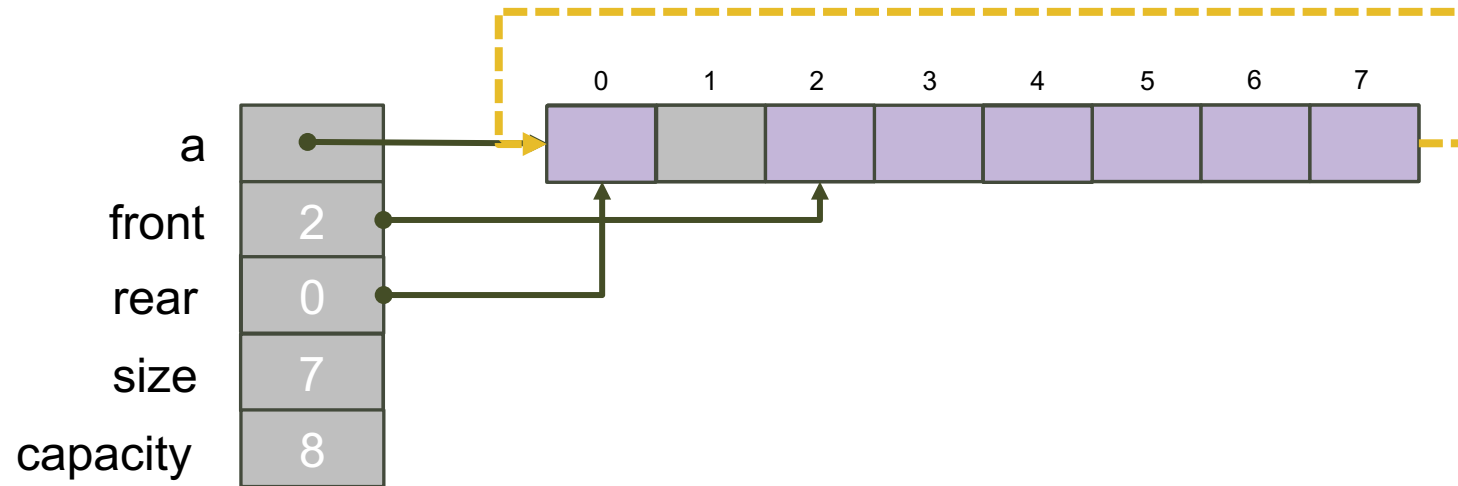
Coda Circolare



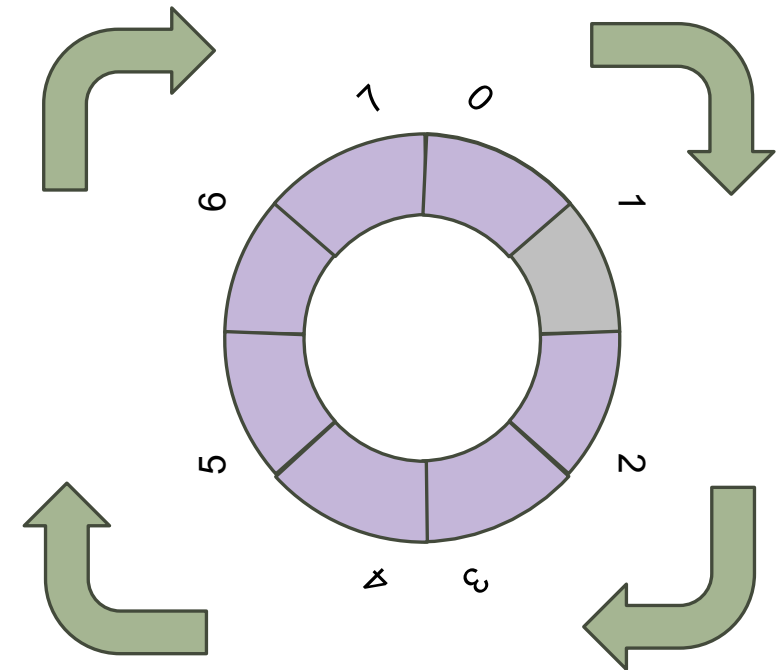
- Coda circolare:
 - Raggiunto `capacity`
 - Gli indici saltano allo 0
 - Esempio: `enqueue()`
 -



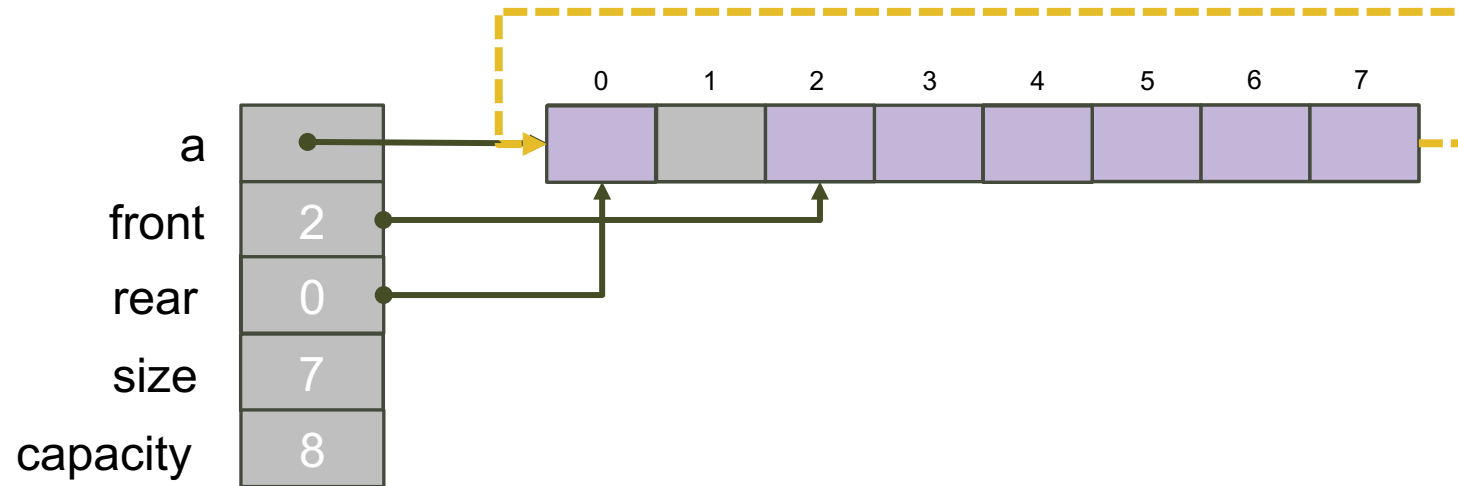
Coda Circolare



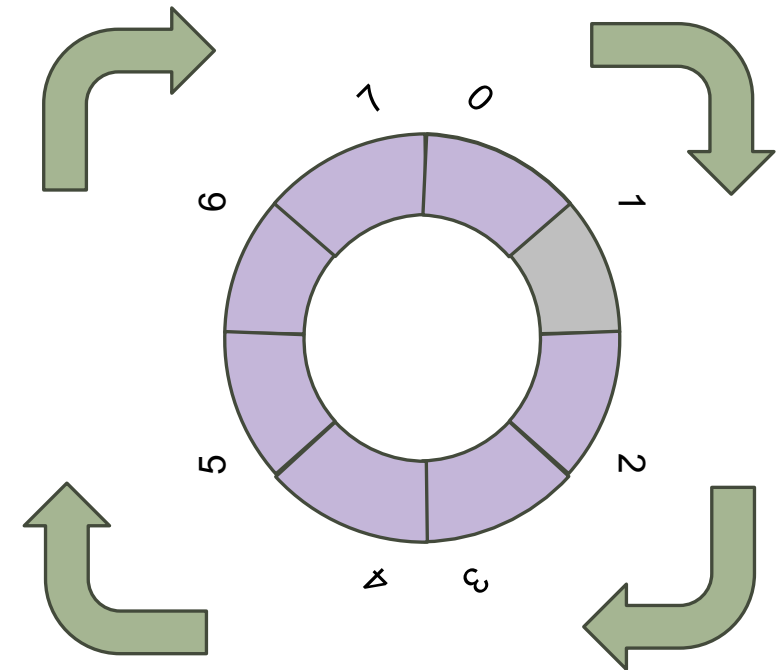
- Coda circolare:
 - Raggiunto `capacity`
 - Gli indici saltano allo 0
 - Esempio: `enqueue()`
 -



Coda Circolare



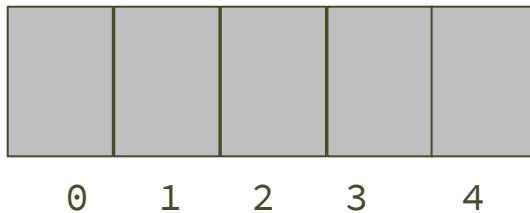
- Coda circolare:
 - Raggiunto `capacity`
 - Gli indici saltano allo 0
 - Esempio: `enqueue()`
 - Come?



Serve uno Strumento: Modulo

- **$N \% k$** : N modulo k, è il resto della divisione intera di N per k.
 - Si usa per calcolare il successore in un array circolare.
- Esempio:
 - Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```



Serve uno Strumento: Modulo

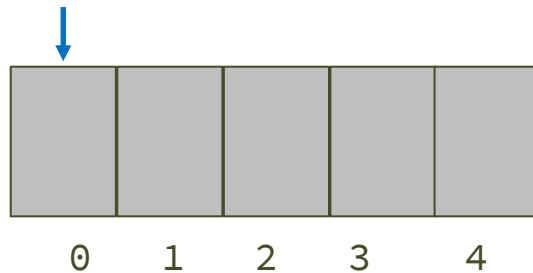
- **$N \% k$** : N modulo k, è il resto della divisione intera di N per k.

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



Serve uno Strumento: Modulo

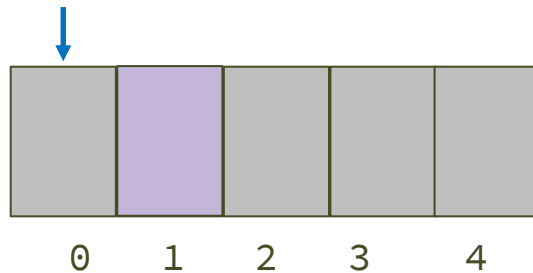
- **$N \% k$** : N modulo k, è il resto della divisione intera di N per k.

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



`successore(0, 5) --> 1`

Serve uno Strumento: Modulo

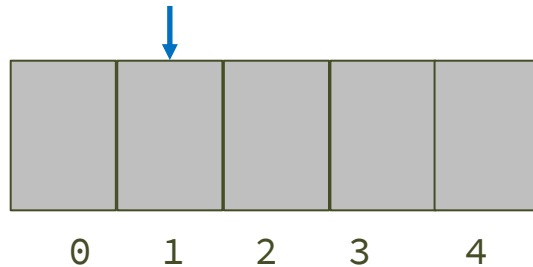
- **$N \% k$** : N modulo k, è il resto della divisione intera di N per k.

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



`successore(0, 5) --> 1`

Serve uno Strumento: Modulo

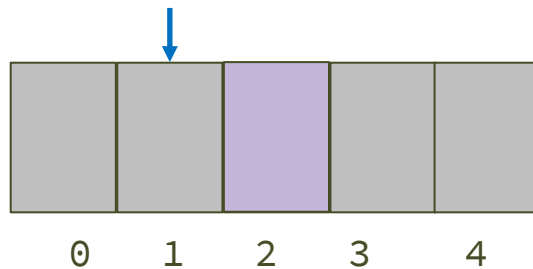
- **$N \% k$** : N modulo k , è il resto della divisione intera di N per k .

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



```
successore(0, 5) --> 1  
successore(1, 5) --> 2
```

Serve uno Strumento: Modulo

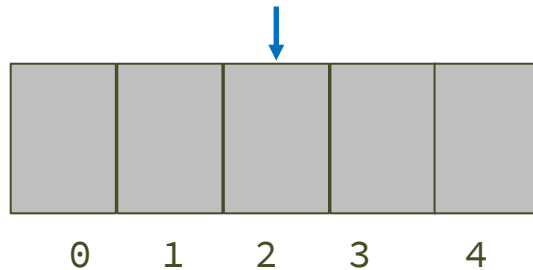
- **$N \% k$** : N modulo k, è il resto della divisione intera di N per k.

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



```
successore(0, 5) --> 1  
successore(1, 5) --> 2
```

Serve uno Strumento: Modulo

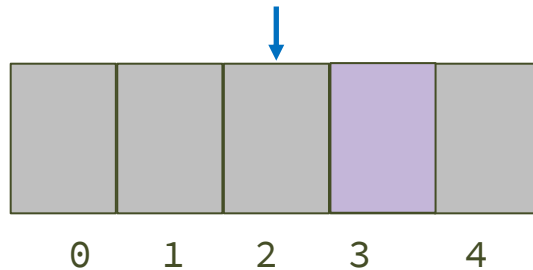
- **$N \% k$** : N modulo k , è il resto della divisione intera di N per k .

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



```
successore(0, 5) --> 1  
successore(1, 5) --> 2  
successore(2, 5) --> 3
```


Serve uno Strumento: Modulo

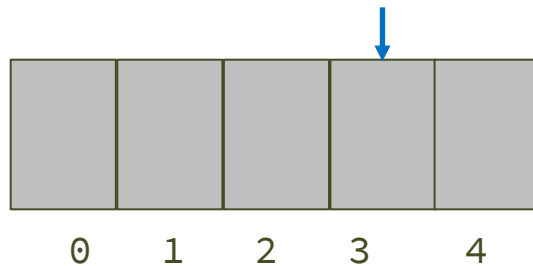
- $N \% k$: N modulo k, è il resto della divisione intera di N per k.

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



```
successore(0, 5) --> 1  
successore(1, 5) --> 2  
successore(2, 5) --> 3
```

Serve uno Strumento: Modulo

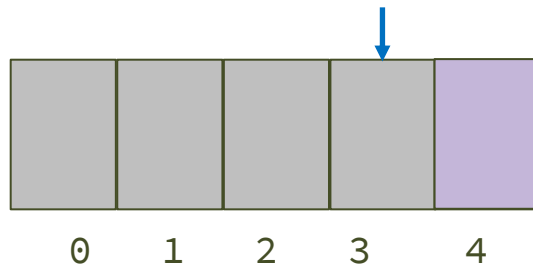
- **$N \% k$** : N modulo k, è il resto della divisione intera di N per k.

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



```
successore(0, 5) --> 1  
successore(1, 5) --> 2  
successore(2, 5) --> 3  
successore(3, 5) --> 4
```

Serve uno Strumento: Modulo

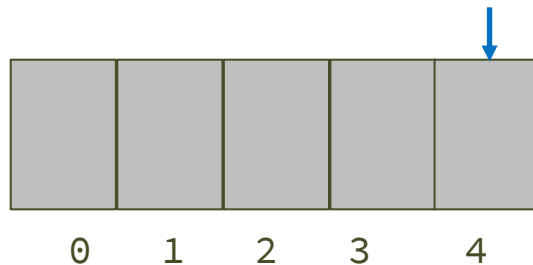
- **$N \% k$** : N modulo k, è il resto della divisione intera di N per k.

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



```
successore(0, 5) --> 1  
successore(1, 5) --> 2  
successore(2, 5) --> 3  
successore(3, 5) --> 4
```

Serve uno Strumento: Modulo

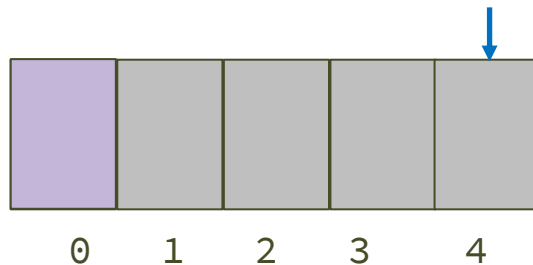
- **$N \% k$** : N modulo k, è il resto della divisione intera di N per k.

- Si usa per calcolare il successore in un array circolare.

```
int successore(i, N) {  
    return ((i+1)%N);  
}
```

- Esempio:

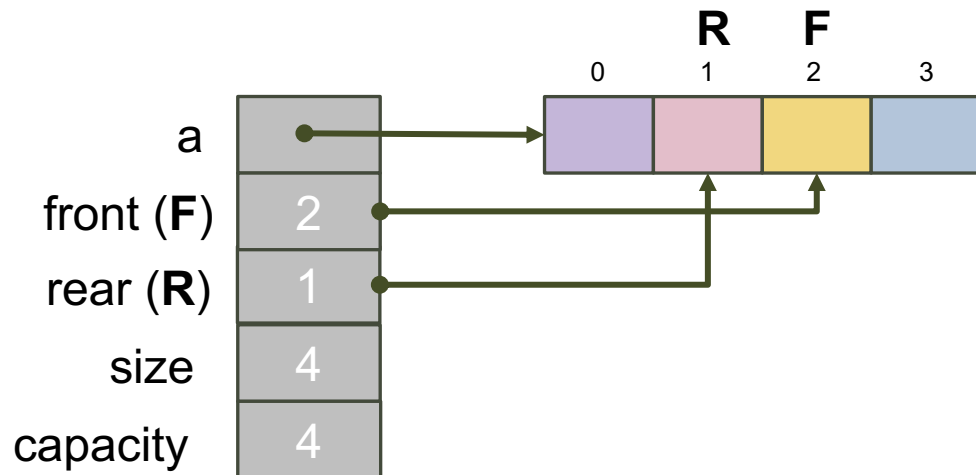
- Supponiamo di avere un array di 5 celle, gli indici vanno da 0 a 4



```
successore(0, 5) --> 1  
successore(1, 5) --> 2  
successore(2, 5) --> 3  
successore(3, 5) --> 4  
successore(4, 5) --> 0
```

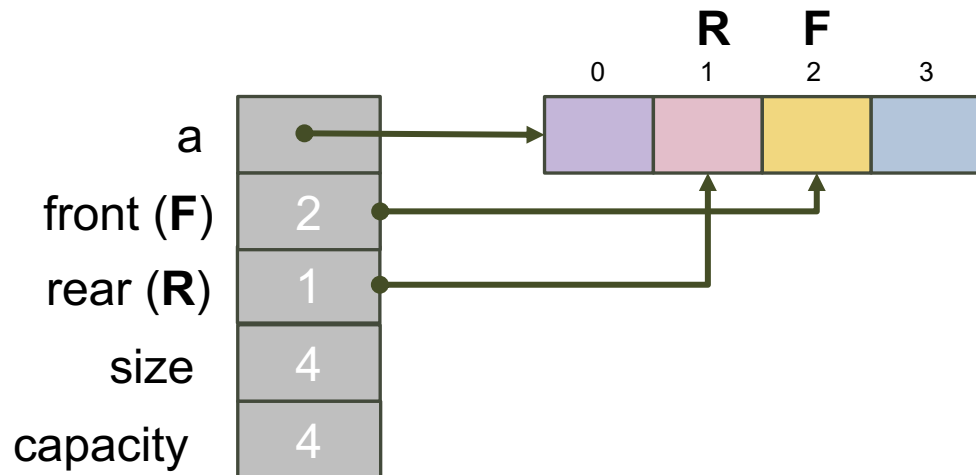
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 -



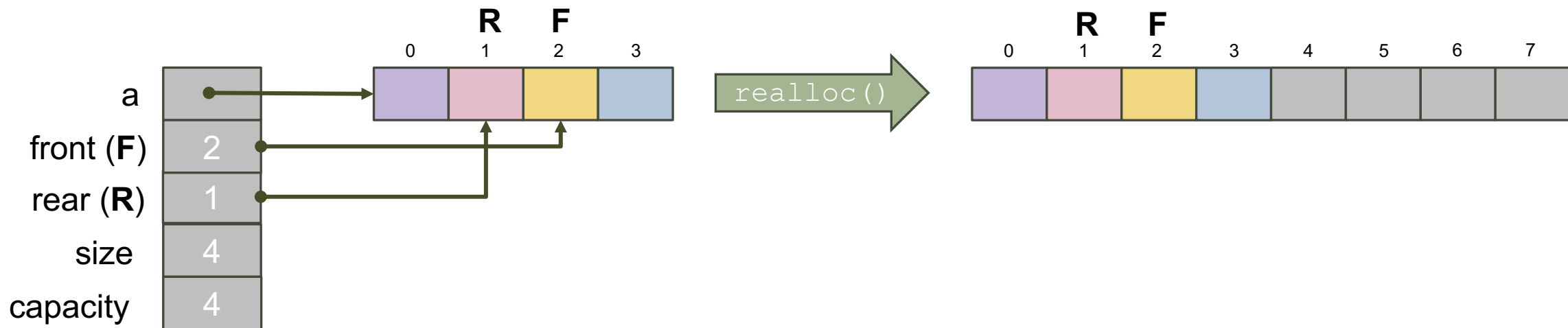
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



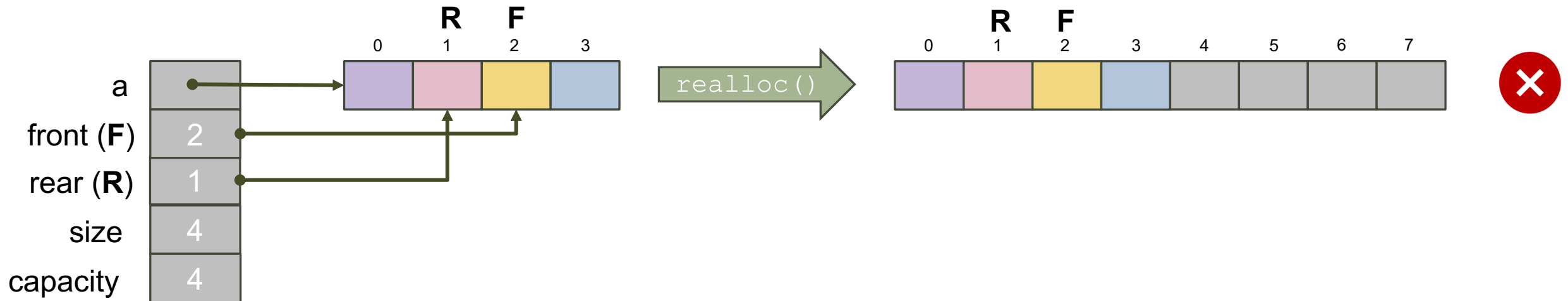
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



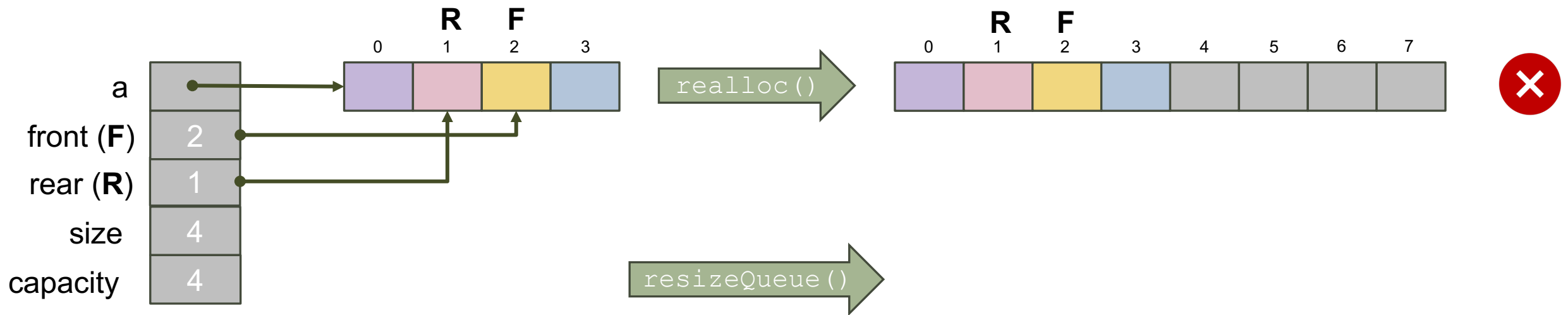
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



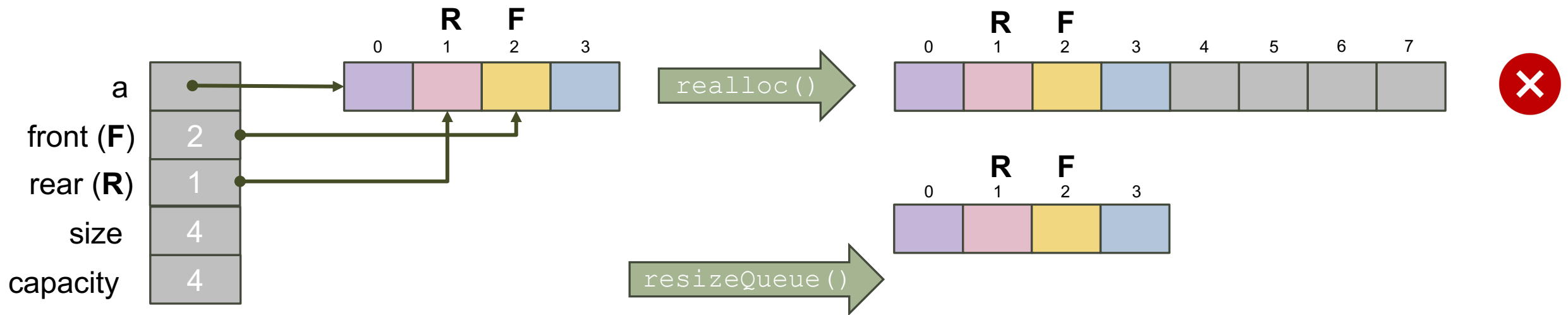
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



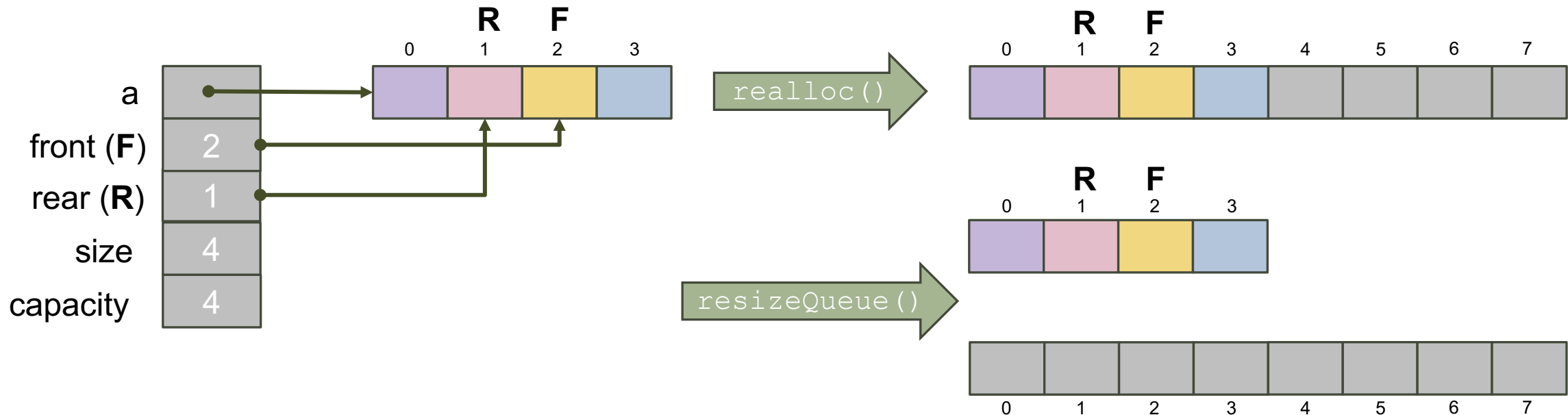
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



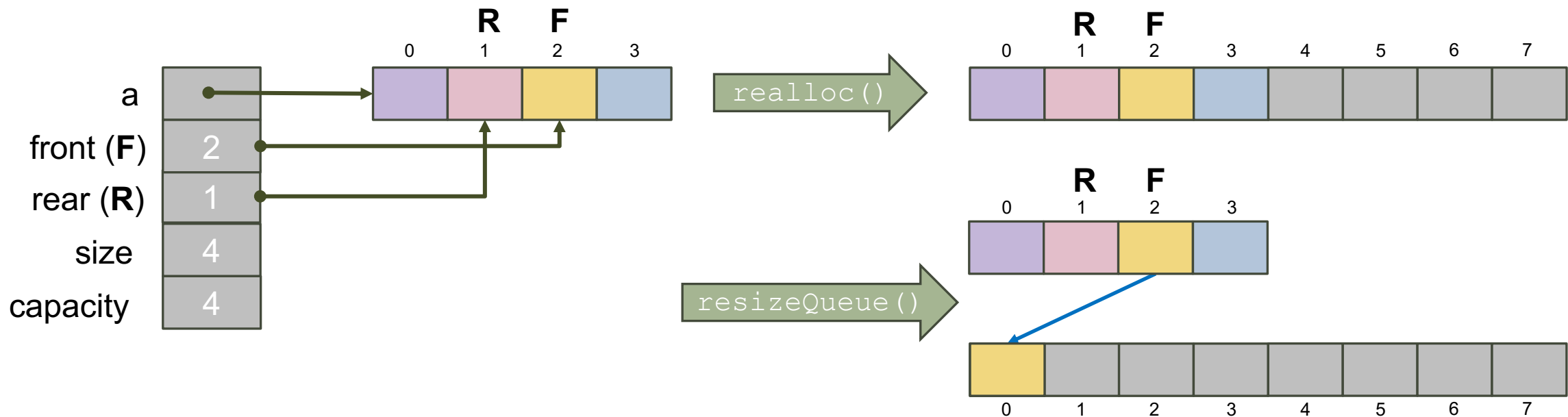
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



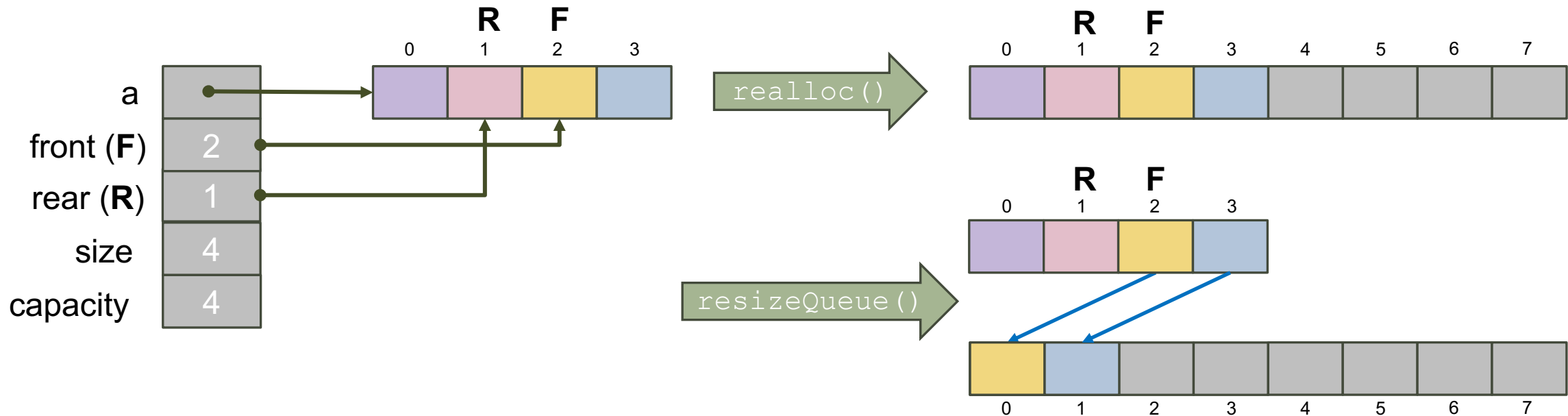
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



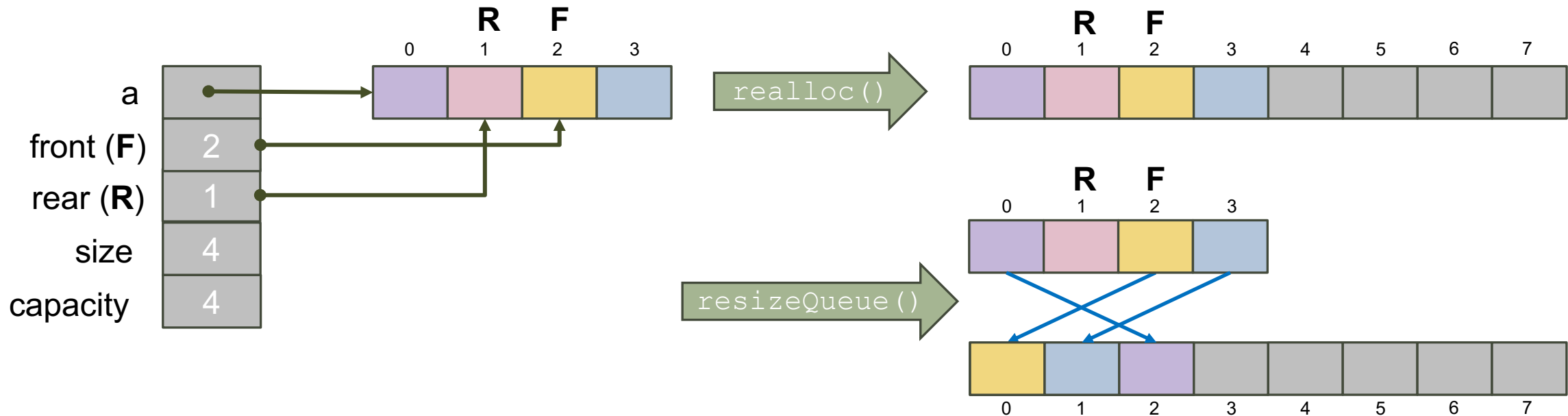
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



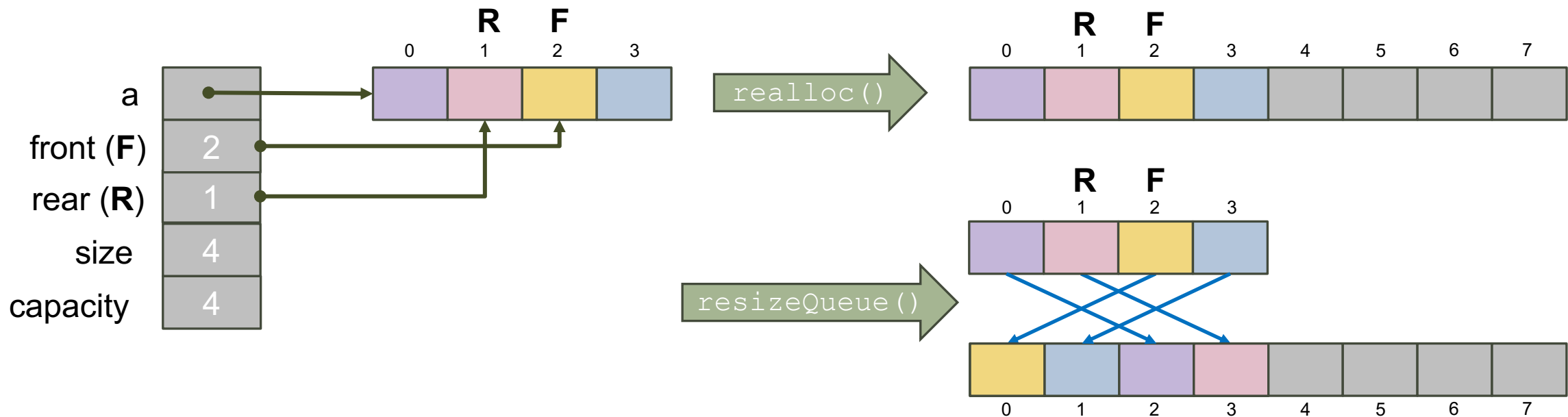
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



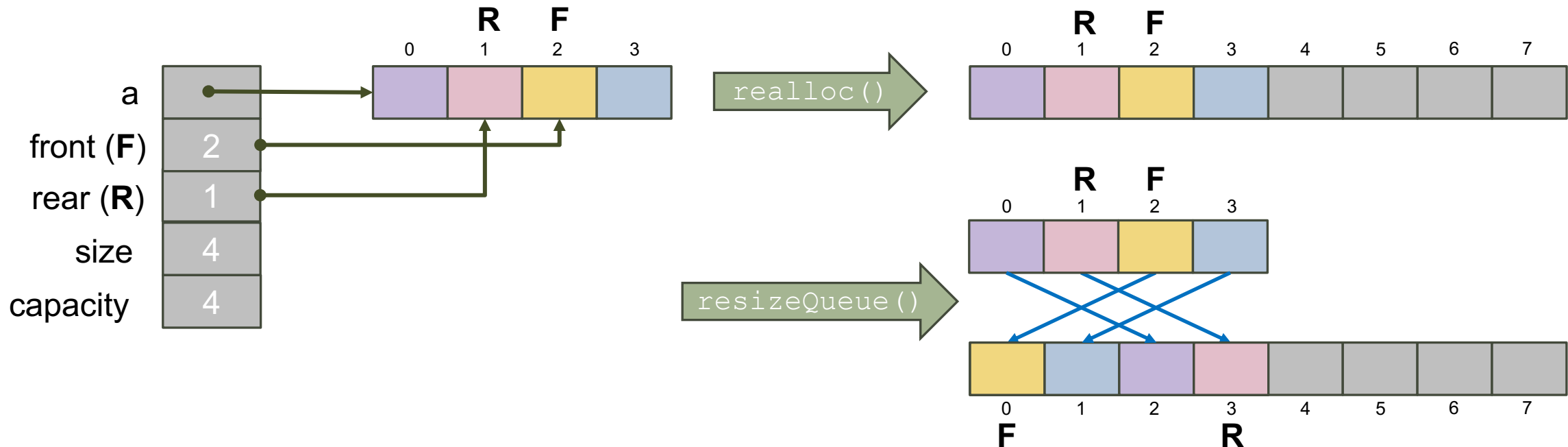
Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



Ridimensionare Array

- Se coda piena:
 - Dobbiamo allocare un nuovo array
 - Possiamo usare realloc?
 - Problema: potrebbe creare un buco dato dalle nuove celle



Esercizio: Coda di char con Array Dinamici

- Completare:
 - `Bool enqueue(CharQueueADT q, char e)`
 - Aggiunge un `char` alla coda
- E implementare le operazioni rimanenti:
 - `CharQueueADT mkStack()`
 - Restituisce una coda vuota
 - `void dsQueue(CharQueueADT *qs)`
 - Distrugge la coda
 - `char dequeue(CharQueueADT q)`
 - Toglie e restituisce il `char` in testa alla coda
 - `_Bool isEmptyQueue(const CharQueueADT q)`
 - Controlla se la coda è vuota
 - `int queueSize(const CharQueueADT q)`
 - Restituisce il numero di elementi nella coda
 - `char peek(CharQueueADT q, int position)`
 - Restituisce l'elemento in posizione (senza toglierlo)
- Tipo e signature definite in `charQueueADT.h`

Alcune Applicazioni delle Queue

- **Buffer:** nei sistemi produttore-consumatore i produttori inseriscono dei dati in una coda, i dati sono estratti nell'ordine FIFO dai consumatori, quando sono pronti a elaborarli
- **Scheduling della CPU:** il sistema operativo mantiene i processi "ready" (che possono eseguire) in una coda che in alcuni sistemi è gestita FIFO
- **Ricerca in ampiezza:** la strategia di ricerca in ampiezza usata nel problem solving si appoggia a una coda FIFO